Python 03: lists and loops : https://bioinfmsc8.bio.ed.ac.uk/AY25_BPSM13.html

---

Quick links:
Click on any ⌂ to get back here!

List-related topics

Loop-related topics

Exercises

Lists : defining a list and getting elements

Useful links
https://developers.google.com/edu/python/
https://docs.python.org/3/tutorial/index.html

Storing multiple values is currently awkward/tedious, as we saw in the exercise from the previous Python session, where we created lots of different variables holding essentially the same kind of values : text strings.

```
# What are the local exon sub-sequences?
local_exon01
'ATCGATCGATCGATCTGAGACTGACTAGTCATAGCTATGCATGTAGCTACTCGATCGATCGA
local_exon02
'GCTATCATCGATCGATATCGATGCATCGACTACTAT'
```

Python's solution is to have **lists** of data, which can store multiple items with values.

Lists start and end with SQUARE brackets **[** *element1, element2* **]** and the elements are separated by commas.

Once we have stored some values in a list, how do we access individual members of that list?

```
# Make an empty list
somecolours1 = []
```

```
# Make a list with some items
somecolours1 = ['red','blue','green']
```

```
# Get a single element by using its index position
# Indexes work just like they did for strings
#
# Remember! start counting from zero, inclusive at the start, exclusive at the end.
somecolours1[0]
```

```
'red'
```

```
somecolours1[1]
```

```
'blue'
```

```
somecolours1[2]
```

```
'green'
```

```
somecolours1[1:]
```

```
['blue', 'green']
```

```
somecolours1[1:3]
```

```
['blue', 'green']
```

```
# Show the first 6 index position elements (even though we know there are only 3)
somecolours1[0:5]
['red', 'blue', 'green']
```

```
# What is in index position 3? Doesn't exist!
somecolours1[3]
Traceback (most recent call last) :
  File '<stdin>', line 1, in <module>
IndexError : list index out of range
```

```
# What is in the last position?
somecolours1[-1]
'green'
```

```
# What is in the penultimate position?
somecolours1[-2]
'blue'
```

```
# Members of the set
somecolours1
['red', 'blue', 'green']
```

```
# Do you get the same as me here? Why?
set(somecolours1)
{ 'green', 'red', 'blue' }
```

```
list(set(somecolours1))
['green', 'red', 'blue']
```

```
sorted(list(set(somecolours1)))
['blue', 'green', 'red']
```

Redundant lists, using a new list

```
somecolours2 = ['blue','green','red','green']
somecolours2
```

```
['blue', 'green', 'red', 'green']
```

```
# How many elements in our list?
len(somecolours2)
```

```
4
```

```
# How many green elements?
somecolours2.count('green')
```

```
2
```

```
# Can we use find() to find the first green element?
somecolours2.find('green')
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'list' object has no attribute 'find'
```

```
# Find the FIRST index position of an element we are interested in
somecolours2.index('green')
```

```
1
```

```
# Find the non-redundant (and sorted) set of elements in the list
set(somecolours2)
```

```
{'green', 'red', 'blue'}
```

```
list(set(somecolours2))
```

```
['green', 'red', 'blue']
```

```
sorted(list(set(somecolours2)))
```

```
['blue', 'green', 'red']
```

```
sorted((set(somecolours2)))
```

```
['blue', 'green', 'red']
```

```
# Variable has unchanged contents
```

```
somecolours2
```

```
['blue', 'green', 'red', 'green']
```

```
# We can use the same name to keep the new version
```

```
Think carefully before doing this...!
```

```
somecolours2 = sorted((set(somecolours2)))
```

```
somecolours2
```

```
['blue', 'green', 'red']
```

List Methods

Here are some other common list `methods`.

- **list.append(item)** : adds a single item to the end of the list
- **list.insert(index, item)** : inserts the item at the given index position, "moving" items to the right
- **list.extend(list2)** : adds the items in list2 to the end of the list. Using `+` or `+=` on a list is similar to using extend()
- **list.index(item)** : searches for the given item from the start of the list and returns its index. Throws a ValueError if the item does not appear (use `in` to check without a ValueError)
- **list.remove(item)** : searches for the first instance of the given item and removes it (throws ValueError if not present)
- **list.sort()** : sorts the list in place; the sorted() function is probably better
- **list.reverse()** : reverses the list in place
- **list.pop(index)** : removes and returns the item at the given index, defaults to the rightmost item, a bit like the opposite of append()

Today we will use many of these, and see some of the more common (easily-made) mistakes we can make.

I have deliberately included the errors because we can always learn from mistakes, and you will be able to refer back to this to see how we corrected the error (or better still, found a different way around the problem) should you need to!

I have also included the errors because Python error messages are useful, so it is a good idea to learn how to interpret them!

Building a longer list : using `append()`

```
# Often we start with an empty list and need to add items to it
# Create a blank list
somecolours1 = []
print(somecolours1)
[]
```

```
# Add a value on to the end of the list
somecolours1.append('red')
print(somecolours1)
['red']
```

```
# We can also concatenate two (or more) lists, just like we did with strings
somecolours1 + ['blue','green']
['red', 'blue', 'green']
```

```
# But this didn't change the contents of original list! Why?
somecolours1
['red']
```

```
# If we try to concatenate a list and a string, we will get an error!
somecolours1 + 'blue'
Traceback (most recent call last) :
  File '<stdin>', line 1, in <module>
TypeError : can only concatenate list (not 'str') to list
```

```
# Ah, we need to append it!
somecolours1.append(['blue','green'])
```

```
# Was that what we wanted? No...not really....!
somecolours1
['red', ['blue', 'green']]
somecolours1[0]
'red'
somecolours1[1]
['blue', 'green']
```

```
# Try to build it up again, first element then...
somecolours1[0] + ['blue','green']
Traceback (most recent call last) :
  File "<stdin>", line 1, in <module>
TypeError : Can't convert 'list' object to str implicitly
```

```
# Try again; not what we want either!!
list(somecolours1[0]) + ['blue','green']
['r', 'e', 'd', 'blue', 'green']
somecolours1[0]
'red'
somecolours1[1]
['blue', 'green']
```

```
# Let's just flatten it...
# using flatten in the pandas module
import pandas
from pandas.core.common import flatten
somecolours1
['red', ['blue', 'green']]
flatten(somecolours1)
<generator object flatten at 0x7fb49c32de60>
list(flatten(somecolours1))
['red', 'blue', 'green']
```

```
# Make a new variable to hold the contents of our flattened list...
colours_fixed = list(flatten(somecolours1))
colours_fixed
['red', 'blue', 'green']
```

```
# What was the more simple alternative?
somecolours1 = []
somecolours1.append('red')
somecolours1
['red']
somecolours1.extend(['blue','green'])
somecolours1
['red', 'blue', 'green']
```

```
# How long is our list?
somecolours2 = ['blue', 'green', 'red', 'green']
somecolours2
['blue', 'green', 'red', 'green']
```

```
# Wrong way to find the length of our list
somecolours2.len()
Traceback (most recent call last) :
  File '<stdin>', line 1, in <module>
AttributeError : 'list' object has no attribute 'len'
```

```
# Correct way to find the length of our list
len(somecolours2)
4
```

```
# A list element
somecolours2[1]
'green'
```

```
# Incorrect way to find the length of a list element
somecolours2[1].len()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'str' object has no attribute 'len'
```

```
# Correct way to find the length of a list element
len(somecolours2[1])
5
```

```
# Correct way to find the length of the non-redundant list
len(set(somecolours2))
```

3

Creating a list by splitting a string

The `split()` method of strings returns a list : remember, that was when we first saw the square brackets `[]` in the last lecture...

```
# Create a variable containing a string
sentence1 = 'one two three four'
sentence1
```

```
'one two three four'
```

```
# Split the string using space as separator (default)
sentence1.split()
```

```
['one', 'two', 'three', 'four']
```

```
# Split the string using the letter t instead!
sentence1.split('t')
```

```
['one ', 'wo ', 'hree four']
```

```
# We could make a string into a list then
# into a string using join
''.join(sentence1.split('t'))
```

```
'one wo hree four'
```

```
# Split a different string using tab as separator (also default)
sentence2 = 'one\ttwo\tthree\tfour'
sentence2.split()
```

```
['one', 'two', 'three', 'four']
```

```
# Split a different string using newline as separator (also default): same result again!
```

```
sentence2 = 'one \n two \n three \n four'
sentence2.split()
```

```
['one', 'two', 'three', 'four']
```

```
# Split the new string using the letter t instead
sentence2.split('t')
```

```
['one\n', 'wo\n', 'hree\nfour']
```

```
# We could make the resulting list into a string
''.join(sentence2.split('t'))
```

```
'one\nwo\nhree\nfour'
```

```
# With a space as joiner
' '.join(sentence2.split('t'))
```

```
'one\n wo\n hree\nfour'
```

```
# But be careful about printing...
print(sentence2.split('t'))
```

```
['one\n', 'wo\n', 'hree\nfour']
```

```
# `join` makes it a string
' '.join(sentence2.split('t'))
```

```
'one\n wo\n hree\nfour'
```

```
# Now we are printing a string, so the `\n` means something different...
print(' '.join(sentence2.split('t')))
```

```
one
 wo
 hree
four
```

```
# Split the new string with a non-present character
sentence2.split('Z')
```

```
['one\ntwo\nthree\nfour']
```

```
# We can't use an empty separator
sentence2.split('')
```

```
Traceback (most recent call last) :
  File '<stdin>', line 1, in <module>
ValueError : empty separator
```

```
# split() doesn't change the value of the string
sentence1
'one two three four'
sentence2
'one\ntwo\nthree\nfour'
```

If we wanted to split a string using multiple split delimiters, we would need to use the regular expression (**re**) approach, which uses the `re` module we briefly mentioned in the last lecture.
How to do this will be covered in a later lecture... Python07 ...so you'll just have to be patient!

Ranges

Another way to create a list is by using `range()`.

This behaved differently in old versions of Python (e.g. 2.x)!

The range type represents an **immutable** (can not be changed) sequence of numbers and is commonly used for looping a specific number of times in loops.

```
# Python 2.x behaviour : automatically produces a list
# With one argument it counts from zero to just before the number : up to but not including
range(10)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
# Our Python 3.12.3 behaviour : produces an object that needs to be made into a list
range(10)
range(0, 10)
type(range(10))
<class 'range'>
```

```
# Is it like a method?
range(10).list()
Traceback (most recent call last) :
  File '<stdin>', line 1, in <module>
AttributeError : 'range' object has no attribute 'list'
```

```
# It is a special kind of object in Python 3.x, so has to be treated differently
list(range(10))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
# Use two arguments to count from the first number up to, but not including, the second
```

```
list(range(5,12))
```

```
[5, 6, 7, 8, 9, 10, 11]
```

```
# Use three arguments to indicate start, stop and step size
```

```
list(range(3,24,4))
```

```
[3, 7, 11, 15, 19, 23]
```

```
# And no, you can't (directly) use a range of letters!
```

```
# This will not give us the alphabet in lower case!
```

```
list(range(str(a),str(z)))
```

```
Traceback (most recent call last) :
  File '<stdin>', line 1, in <module>
NameError : name 'a' is not defined
```

```
# But you can if you import the string module ....
```

```
import string
```

```
help(string)
```

```
NAME
    string - A collection of string constants.

MODULE REFERENCE
    https://docs.python.org/3.12/library/string.html

    The following documentation is automatically generated from the Python
    source files.  It may be incomplete, incorrect or include features that
    are considered implementation detail and may vary between Python
    implementations.  When in doubt, consult the module reference at the
    location listed above.

DESCRIPTION
    Public module variables:

    whitespace -- a string containing all ASCII whitespace
    ascii_lowercase -- a string containing all ASCII lowercase letters
    ascii_uppercase -- a string containing all ASCII uppercase letters
    ascii_letters -- a string containing all ASCII letters
    digits -- a string containing all ASCII decimal digits
    hexdigits -- a string containing all ASCII hexadecimal digits
    octdigits -- a string containing all ASCII octal digits
    punctuation -- a string containing all ASCII punctuation characters
    printable -- a string containing all ASCII characters considered printable

CLASSES
    builtins.object
        Formatter
        Template

    class Formatter(builtins.object)
     |  Methods defined here:
     |
     |  check_unused_args(self, used_args, args, kwargs)
     |
     |  convert_field(self, value, conversion)
     |
     |  format(self, format_string, /, *args, **kwargs)
     |
     |  format_field(self, value, format_spec)
     |
     |  get_field(self, field_name, args, kwargs)
     |      # given a field_name, find the object it references.
```

```
..... (other stuff)
```

```
# Let's try it with the lowercase ascii set again
```

```
ascii_lowercase
```

```
Traceback (most recent call last) :
  File '<stdin>', line 1, in <module>
NameError : name 'ascii_lowercase' is not defined
```

```
# We need to indicate which module it comes from!
```

```
string.ascii_lowercase
```

```
'abcdefghijklmnopqrstuvwxyz'
```

```
# We can evaluate bits of it
```

```
string.ascii_lowercase.startswith('abc')
```

```
True
```

```
string.ascii_lowercase.startswith('bcd')
```

```
False
```

```
string.ascii_lowercase.endswith('xyz')
```

```
True
```

```
string.ascii_lowercase.endswith('z')
```

```
True
```

```
string.ascii_lowercase[-3:]
```

```
'xyz'
```

```
string.ascii_lowercase[-3:].endswith('yz')
```

```
True
```

```
# We can convert the string to a list
```

```
list(string.ascii_lowercase)
```

```
['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q
```

```
# This example is pointless, but doable!
```

```
list(string.ascii_lowercase.upper())
```

```
['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q
```

```
# An index can be used
```

```
list(string.ascii_lowercase)[0:3]
```

```
['a', 'b', 'c']
```

```
# Could we use a non-sequential index without using a loop?
non - sequential = 4,5,9
```

```
  File "<stdin>", line 1
SyntaxError : cannot assign to operator
```

```
nonsequential = 4,5,9
nonsequential
(4, 5, 9)
list(string.ascii_lowercase)[nonsequential]
```

```
Traceback (most recent call last) :
  File '<stdin>', line 1, in <module>
TypeError : list indices must be integers, not tuple
```

A **tuple** is a sequence of immutable (unchangeable) Python objects : in other words, tuples cannot be changed (unlike lists) and tuples use `( )` whereas lists use `[ ]`

For more info on tuples and other in-built data types, see https://www.w3schools.com/python/python_tuples.asp and related/linked pages.

```
# This is a tuple
nonsequential = 4,5,7,9
nonsequential
(4, 5, 7, 9)
```

```
# It DOES have an index
nonsequential[1]
5
```

```
# We can't change the contents of a tuple
nonsequential[1] = 10
```

```
Traceback (most recent call last) :
  File '<stdin>', line 1, in <module>
TypeError : 'tuple' object does not support item assignment
```

```
# We can't append to the contents of a tuple
nonsequential[4] = 10
```

```
Traceback (most recent call last) :
  File '<stdin>', line 1, in <module>
TypeError : 'tuple' object does not support item assignment
```

```
# Could we use a range to get the first three letters of the ascii_lowercase string?
list(string.ascii_lowercase)[range(0,3)]
Traceback (most recent call last) :
  File '<stdin>', line 1, in <module>
TypeError : list indices must be integers, not range

# Would using a list to subset a list work?
list(range(0,3))
[0, 1, 2]
list(string.ascii_lowercase)[list(range(0,3))]
Traceback (most recent call last) :
  File '<stdin>', line 1, in <module>
TypeError : list indices must be integers, not list
# What did the error message say!? list indices must be integers

# Might this work then?
list(string.ascii_lowercase)[int(list(range(0,3)))]
Traceback (most recent call last) :
  File '<stdin>', line 1, in <module>
TypeError : int() argument must be a string or a number, not 'list'

# We can print out a list if it is strings
somecolours2
['blue', 'green', 'red', 'green']

# Print using ', ' to separate the list items
print(', '.join(somecolours2))
blue, green, red, green

# But not directly if it is integers
a=list(range(0,3))
```

```
print(','.join(a))
```

```
Traceback (most recent call last) :
  File '<stdin>', line 1, in <module>
TypeError : sequence item 0 : expected str instance, int found
```

```
# Things we can do
print(a)
[0, 1, 2]
print(a[0])
0
str(a)
'[0, 1, 2]'
print(str(a))
[0, 1, 2]
# Brackets are important!
print(str(a[0]))
0
print(str(a)[0])
[
```

```
str(a)[1:-1]
'0, 1, 2'
print(str(a)[1:-1])
0, 1, 2
```

```
# It's a string, so we could do 'string things' if we wanted
str(a).replace(', ' , ' then ')
'[0 then 1 then 2]'
str(a)[0 :-1].replace(', ' , ' then ')
'[0 then 1 then 2'
str(a)[1 :-1].replace(', ' , ' then ')
'0 then 1 then 2'
```

Manipulating lists : sorting or reversing a list

```
# Make a somecolours1 list
somecolours1 = ['blue','red','green']
somecolours1
['blue', 'red', 'green']
```

```
# Call the sort() method to make them alphabetical in order
# Note that we are not creating a new list object
# The original list IS changed
somecolours1.sort()
somecolours1
['blue', 'green', 'red']
```

```
# the sort() method doesn't work without the () brackets!
somecolours1.sort
<built-in method sort of list object at 0x7f212fc8b688>
```

```
# Make a integer list and sort it
numbs = [3,8,4,6,9,1,5,7,4,2,6,8,5,3]
numbs.sort()
numbs
[1, 2, 3, 3, 4, 4, 5, 5, 6, 6, 7, 8, 8, 9]
```

```
# Reverse the sort order of the lists
# Text strings
somecolours1 = ['blue','red','green']
somecolours1
['blue', 'red', 'green']
somecolours1.reverse()
somecolours1
['green', 'red', 'blue']
```

```
# Non-redundant set
set(somecolours1)
{'blue', 'green', 'red'}
list(set(somecolours1))
['blue', 'green', 'red']
```

```
# Can't do this though!
set(somecolours1.reverse())
Traceback (most recent call last) :
  File '<stdin>', line 1, in <module>
TypeError : 'NoneType' object is not iterable
```

```
# Using join
```

```
s='-'
s.join(somecolours1)
'green-red-blue'
s='|'
s.join(somecolours1)
'green|red|blue'
s=' then '
s.join(somecolours1)
'green then red then blue'
```

```
# Integer values
numbs
[1, 2, 3, 3, 4, 4, 5, 5, 6, 6, 7, 8, 8, 9]
numbs.reverse()
numbs
[9, 8, 8, 7, 6, 6, 5, 5, 4, 4, 3, 3, 2, 1]
```

```
len(numbs)
14
len(set(numbs))
9
```

```
# Non-redundant (ordered) set
numbs
[9, 8, 8, 7, 6, 6, 5, 5, 4, 4, 3, 3, 2, 1]
set(numbs)
{1, 2, 3, 4, 5, 6, 7, 8, 9}
list(set(numbs))
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
# Will these work? NO!
numbs.reverse().sort()
Traceback (most recent call last) :
  File '<stdin>', line 1, in <module>
AttributeError : 'NoneType' object has no attribute 'sort'
```

```
numbs.sort().reverse()
```

```
Traceback (most recent call last) :
  File '<stdin>', line 1, in <module>
AttributeError : 'NoneType' object has no attribute 'reverse'
```

```
# Can we use join?
```

```
s=''
```

```
s.join(numbs)
```

```
Traceback (most recent call last) :
  File '<stdin>', line 1, in <module>
TypeError : sequence item 0 : expected str instance, int found
```

```
# We know how to deal with this!
```

```
s.join(str(numbs))
```

```
'[9, 8, 8, 7, 6, 6, 5, 5, 4, 4, 3, 3, 2, 1]'
```

```
s.join(str(numbs)[1 :-1])
```

```
'9, 8, 8, 7, 6, 6, 5, 5, 4, 4, 3, 3, 2, 1'
```

```
s.join(str(numbs)[1 :-1]).replace(', ','')
```

```
'98876655443321'
```

Warning, slightly (very?) scary content approaching...

```
# Make a mixed list comprising strings and integers
```

```
somecolours1
```

```
['green', 'red', 'blue']
```

```
somecolours1_and_numbs = somecolours1 + numbs
```

```
somecolours1_and_numbs
```

```
['green', 'red', 'blue', 9, 8, 8, 7, 6, 6, 5, 5, 4, 4, 3, 3, 2, 1]
```

```
# We can't sort the combined list, as the types are different
```

```
somecolours1_and_numbs.sort()
```

```
Traceback (most recent call last) :
  File '<stdin>', line 1, in <module>
TypeError : unorderable types : int() < str()
```

```
# Can we make the integers into a string then
# concatenate them onto the somecolours1 list?
```

```
# We could make the whole list into an apparent single string
```

```
str(numbs)
```

```
'[9, 8, 8, 7, 6, 6, 5, 5, 4, 4, 3, 3, 2, 1]'
```

```
# But this still won't work as the types are different
somecolours1_and_numbs = somecolours1 + str(numbs)
```

```
Traceback (most recent call last) :
  File '<stdin>', line 1, in <module>
TypeError : can only concatenate list (not 'str') to list
```

```
# So, let's make the 'stringed' integers list back into a list
somecolours1_and_numbs = somecolours1 + list(str(numbs))
somecolours1_and_numbs
```

```
['green', 'red', 'blue', '[', '9', ',', ' ', '8', ',', ' ', '8', ',', ' ', '7', ',', ' ', '6', ',', ' ', '6', ',', ' ', '5', ',', ' ', '5', ',', ' ', '4', ',', ' ', '4', ',', ' ', '3', ',', ' ', '3', ',', ' ', '2', ',', ' ', '1', ']']
```

```
# That works, but wasn't what we wanted, I don't think!
# We need to use something a bit more complex...
numbs
```

```
[9, 8, 8, 7, 6, 6, 5, 5, 4, 4, 3, 3, 2, 1]
```

```
# We want to take each element e of the list, and
# for each element in the list
# join it as a string to the next element with separator s which we set as a blank
s=''
s.join(str(e) for e in numbs)
```

```
'98876655443321'
```

```
# Needs to be made into a list again so that we can make the combined list
list(s.join(str(e) for e in numbs))
```

```
['9', '8', '8', '7', '6', '6', '5', '5', '4', '4', '3', '3', '2', '1']
```

```
# So now let's try to combine the lists again
somecolours1_and_numbs = somecolours1 + list(s.join(str(e) for e in numbs))
somecolours1_and_numbs
```

```
['green', 'red', 'blue', '9', '8', '8', '7', '6', '6', '5', '5', '4', '4', '3', '3', '2', '1']
# Right result! This looks much more like what we had in mind!
```

```
# Now sort the list contents : 'numbers' come first, then letters
somecolours1_and_numbs.sort()
somecolours1_and_numbs
['1', '2', '3', '3', '4', '4', '5', '5', '6', '6', '7', '8', '8', '9', 'blue', 'green', 'red']
```

```
# We can reverse it too
somecolours1_and_numbs.reverse()
somecolours1_and_numbs
['red', 'green', 'blue', '9', '8', '8', '7', '6', '6', '5', '5', '4', '4', '3', '3', '2', '1']
```

```
# But watch out...!
# These are OK...
set(somecolours1_and_numbs)
{'green', '4', 'red', 'blue', '7', '1', '9', '6', '5', '8', '2', '3'}
set.intersection(set(somecolours1_and_numbs),set(somecolours1))
{'green', 'red', 'blue'}
set(somecolours1_and_numbs).intersection(somecolours1)
{'blue', 'green', 'red'}
```

```
# This is not...
set(somecolours1_and_numbs).intersection(numbs)
set()
```

```
# But it is now, because all are strings!
set(somecolours1_and_numbs).intersection(list(s.join(str(e) for e in numbs)))
{'4', '7', '1', '9', '6', '5', '8', '2', '3'}
```

```
# This works....
somecolours1[0] in somecolours1_and_numbs
True
```

```
# This gives the wrong answer...
numbs[0] in somecolours1_and_numbs
False
```

```
# But will give the right answer now!
numbs[0]
9
str(numbs[0]) in somecolours1_and_numbs
True
```

... end of definitely very scary content....

So... watch out for the different data types : strings and integers will work together, but it can be very tricky sometimes!

I have deliberately shown you many of the more common mistakes... Hopefully you won't make the same mistakes too, but if you do, you will be able to refer back to this to see how we corrected the error (or better still, found a different way around the problem)...

Loops : doing the 'same stuff' many times

```
# Make another colours list : the somecolours3 list
somecolours3 = ['yellow','purple','black','pink','yellow','blue','red','green']
```

```
# Single item
one_colour = somecolours3[1]
one_colour
'purple'
```

```
# Every second item
somecolours3[ : :2]
['yellow', 'black', 'yellow', 'red']
```

```
# Every third item
somecolours3[ : :3]
['yellow', 'pink', 'red']
```

The notations we used above are called **slicing**, and there is a good description of the HUGE number of possible things we can use here (asked 16 years, 8 months ago Modified 19 days ago Viewed 3.2m times) and this one too (less so, more pretty graphics....)

```
# How do we do something for every element of a list?
for colour_item in somecolours3 :
    print(colour_item)
```

```
# Python treats the blank command line as a 'Do it now, please' command
# The output from the above three lines is :
yellow
purple
black
pink
yellow
blue
red
green
```

Different ways to do string formatting

1. https://www.learnpython.org/en/String_Formatting
2. https://docs.python.org/3.6/library/stdtypes.html#str.format
3. https://realpython.com/python-f-strings/#f-strings-a-new-and-improved-way-to-format-strings-in-python

Using string formatting #1

https://www.learnpython.org/en/String_Formatting

```
for colour_item in somecolours3 :
     "%s" %(colour_item)
```

```
'yellow'
'purple'
'black'
'pink'
'yellow'
'blue'
'red'
'green'
```

Using string formatting #2

https://docs.python.org/3.6/library/stdtypes.html#str.format

```
for colour_item in somecolours3 :
    "{0}".format(colour_item)
```

```
'yellow'
'purple'
'black'
'pink'
'yellow'
'blue'
'red'
'green'
```

Using string formatting #3

https://realpython.com/python-f-strings/#f-strings-a-new-and-improved-way-to-format-strings-in-python

```
for colour_item in somecolours3 :
     f"{colour_item}"
```

```
'yellow'
'purple'
'black'
'pink'
'yellow'
'blue'
'red'
'green'
```

Using string formatting #3

```
for colour_item in somecolours3 :
     f"The next colour is {colour_item}"
```

```
'The next colour is yellow'
'The next colour is purple'
'The next colour is black'
'The next colour is pink'
'The next colour is yellow'
'The next colour is blue'
'The next colour is red'
'The next colour is green'
```

Using string formatting #2 (variables can be positional)

```
count=0
for colour_item in somecolours3 :
     count+=1
     "Colour {1} is {0}".format(count, colour_item)
```

```
'Colour yellow is 1'
'Colour purple is 2'
'Colour black is 3'
'Colour pink is 4'
'Colour yellow is 5'
'Colour blue is 6'
'Colour red is 7'
'Colour green is 8'
```

Using string formatting #3

```
count=0
for colour_item in somecolours3 :
     count+=1
     f"Colour {count} is {colour_item}"
```

```
'Colour 1 is yellow'
'Colour 2 is purple'
'Colour 3 is black'
'Colour 4 is pink'
'Colour 5 is yellow'
'Colour 6 is blue'
'Colour 7 is red'
'Colour 8 is green'
```

```
for colour_item in somecolours3 :
     print(colour_item)
     if colour_item == 'black' :
          f'My favourite colour is {colour_item}'
     if colour_item == 3 :
          f'My favourite number is {colour_item}'
     f'This line is not in either if block'

f'This line is not in the for loop'
```

```
yellow
'This line is not in either if block'
purple
'This line is not in either if block'
black
'My favourite colour is black'
'This line is not in either if block'
pink
'This line is not in either if block'
yellow
'This line is not in either if block'
blue
'This line is not in either if block'
red
'This line is not in either if block'
green
'This line is not in either if block'
```

f'This line is not in the for loop'

'This line is not in the for loop'

Important things

- `colour_item` is a new variable created inside the loop
- the `for` and the `if` lines both end with a colon `:`
- the body of the `for` loop is indented (**tabs or spaces but not both!**)
- the body of the `if` block of code is indented (tabs or spaces but not both!)
- the value of variable `colour_item` is set to each list element in the `for` loop as it goes through each element in the list one by one until it has done them all

---

Loop through the somecolours3 list

```
for colour_item in somecolours3 :
  print('This time, the colour_item variable is ' + colour_item)
  f'formatted way : the colour_item variable is {colour_item}'
```

```
This time, the colour_item variable is yellow
'formatted way : the colour_item variable is yellow'
This time, the colour_item variable is purple
'formatted way : the colour_item variable is purple'
This time, the colour_item variable is black
'formatted way : the colour_item variable is black'
This time, the colour_item variable is pink
'formatted way : the colour_item variable is pink'
This time, the colour_item variable is yellow
'formatted way : the colour_item variable is yellow'
This time, the colour_item variable is blue
'formatted way : the colour_item variable is blue'
This time, the colour_item variable is red
'formatted way : the colour_item variable is red'
This time, the colour_item variable is green
'formatted way : the colour_item variable is green'
```

Example of a simple programme with a code formatting problem

somecolours1

['green', 'red', 'blue']

```
for colour_item in somecolours1 :
    f'{colour_item}'
     f'{colour_item}  again'
```

```
  File '<stdin>', line 3
    f'{colour_item}  again'
    ^
IndentationError : unexpected indent
```

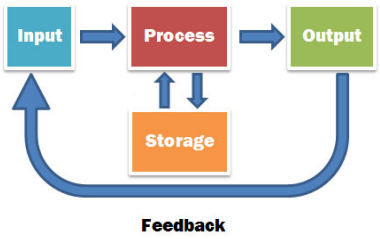
Example of a simple programme with correct formatting

```
for colour_item in somecolours1 :
      f'{colour_item}'
      f'{colour_item}  again'
```

```
'green'
'green  again'
'red'
'red  again'
'blue'
'blue  again'
```

A more complicated loop example

The previous example was a fairly simple example. The loop body can contain all code that we might want to apply to each member of the list.



Simple workflow plan v_1.0.0

START
My INPUT is a series of header text strings in a list
ACTION/PROCESS/OUTPUTS
For each header

- create a new file with the same name as the header
- then write the length of the header to the file
- then print the first three letters of the header in upper case.

CHECK IT'S OK
END

```
# Workflow programme v_1.0.0 and report
# Written by s123456 on Tues 28 October 2025
# Take a list and make files from each element, writing out the length to the file,
# and printing the first 3 characters in upper case
```

```
# Step 1 : Generate the input list
somecolours4 = ['blue','red','green']
```

```
# Step 2 : Loop through the list
# the loop body contains all the actions required
for colour_item in somecolours4 :
    myfile = open(colour_item + '.txt', 'w')
    myfile.write(str(len(colour_item)) + '\n')
    myfile.close()
    f'{colour_item[0:3].upper()}'
```

```
# This was the output
2
'BLU'
2
'RED'
2
'GRE'
```

```
# Check the written files, use a loop for this too
for colour_item in somecolours4 :
    f'Colour list item {colour_item} was
      {open(colour_item + ".txt").read()} characters long.'
```

```
  File "<stdin>", line 2
    f'Colour list item {colour_item} was
                                        ^
SyntaxError: EOL while scanning string literal
```

```
{open(colour_item + ".txt").read()} characters long.'
```

```
  File "<stdin>", line 1
    {open(colour_item + ".txt").read()} characters long.'
    ^
IndentationError: unexpected indent
```

```
# Ooops, try again, with line continuation...
for colour_item in somecolours4 :
    f'Colour list item {colour_item} was \
      {open(colour_item + ".txt").read()} characters long.'
```

```
'Colour list item blue was       4\n characters long.'
'Colour list item red was       3\n characters long.'
'Colour list item green was       5\n characters long.'
```

```
# Try again, with line continuation, back to left hand side
for colour_item in somecolours4 :
    f'Colour list item {colour_item} was \
{open(colour_item + ".txt").read()} characters long.'
```

```
'Colour list item blue was 4\n characters long.'
'Colour list item red was 3\n characters long.'
'Colour list item green was 5\n characters long.'
```

```
# Check the written files, stripping the '\n'
```

```
for colour_item in somecolours4 :
    f'Colour list item {colour_item} was \
{open(colour_item + ".txt").read().rstrip()} characters long.'
```

This is the output now

```
'Colour list item blue was 4 characters long.'
'Colour list item red was 3 characters long.'
'Colour list item green was 5 characters long.'
```

Loops of strings

We can make a loop that uses a string as if it were a list : each character will behave like an individual element.

```
# Strings can pretend to be lists...
dna = 'agcacgacgtagtgatcggcta'
```

```
# each character is an element; each element is a character
for base in dna :
    f'{base}'
```

```
'a'
'g'
'c'
'a'
'c'
'g'
'a'
'c'
'g'
't'
'a'
'g'
't'
'g'
'a'
't'
'c'
'g'
'g'
'c'
't'
'a'
```

Warning!

It's VERY easy to do this by accident so if you see individual characters, check you're looping through the right thing!

Using loops to read file contents

We can treat a file object/connection as if it were a list, so each line in the file is treated as a list element.

This is how we can process the contents of a file in a very efficient way : you will (**SHOULD**) find yourself almost always wanting to do things this way (or something very similar!).

```
# File objects/connections can 'pretend' to be lists
```

```
# Import the modules we need
import os, subprocess
```

```
# Check to see if the file we want is there (I made it earlier)
os.path.exists('test.txt')
```

```
True
```

```
# Show the contents so we know what is there to start with!
subprocess.call('cat test.txt', shell=True)
```

```
abcdefgh
ijklmnop
0123456789
0
0
```

```
# We know how to make a connection... I've called it fcon
fcon = open('test.txt')
# ... and read contents (whole file...)
fcon.read()
```

```
'abcdefgh\nijklmnop\n0123456789\n0\n'
```

```
fcon.read()
''
```

```
# We can do it line by line...
fcon = open('test.txt')
fcon.readline()
'abcdefgh\n'
fcon.readline()
'ijklmnop\n'
fcon.readline()
'0123456789\n'
fcon.readline()
'0\n'
# Nothing left
fcon.readline()
''
```

This is really cool because when our `fcon.readline()` returns an empty string, we know that the end of the file has been reached! A blank line is represented by '\n', a string containing only a single newline character.

If we wanted to read all the lines of a file as a list we can also just use:

```
list(open('test.txt'))
['abcdefgh\n', 'ijklmnop\n', '0123456789\n', '0\n']
```

or

```
open('test.txt').readlines()
['abcdefgh\n', 'ijklmnop\n', '0123456789\n', '0\n']
```

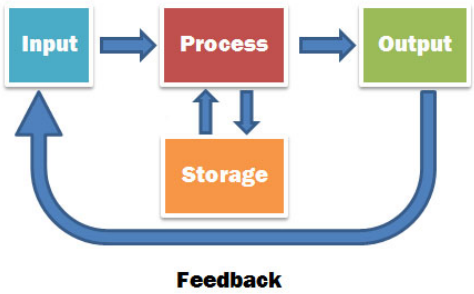
```
# Using the object (connection)...
# Note that in this example we have NOT actually explicitly read() the file contents!
# each list element is a line
count=0
my_file = open('test.txt')
for eachline in my_file :
    count+=1
    line_length = len(eachline.rstrip('\n'))
    'We are processing line {} of the file'.format(count)
```

Give feedback
Ask for help

```
    f'There are {line_length} characters on the line'
```

```
'We are processing line 1 of the file'
'There are 8 characters on the line'
'We are processing line 2 of the file'
'There are 8 characters on the line'
'We are processing line 3 of the file'
'There are 10 characters on the line'
'We are processing line 4 of the file'
'There are 1 characters on the line'
```

Your turn : Exercises time!




```
# Make a directory within your git repository area for today's work,
# and change directory to it.
```

```
mkdir ~/Exercises/Lecture13
```

```
cd ~/Exercises/Lecture13
```

edirect

This is the software that you installed at the end of Lecture 7.

```
# Check the esearch programme works for you
```

```
esearch -h
```

```
esearch 24.7

Query Specification

  -db            Database name
  -query         Query string

Spell Check

  -spell         Correct misspellings in query
```

```
Query Translation

  -translate     Show automatic term mapping
  -component     Individual term mapping items

Document Order

  -sort          Result presentation order

Sort Choices by Database

  gene           Chromosome, Gene Weight, Name, Relevance

  geoprofiles    Default Order, Deviation, Mean Value, Outliers, Subgroup Effect

  pubmed         First Author, Journal, Last Author, Pub Date, Recently Added,
                 Relevance, Title

  (sequences)    Accession, Date Modified, Date Released, Default Order,
                 Organism Name, Taxonomy ID

  snp            Chromosome Base Position, Default Order, Heterozygosity,
                 Organism, SNP_ID, Success Rate

Note

  All efilter shortcuts can also be used with esearch

Examples

  esearch -db pubmed -query "opsin gene conversion OR tetrachromacy"

  esearch -db pubmed -query "Garber ED [AUTH] AND PNAS [JOUR]"

  esearch -db nuccore -query "MatK [GENE] AND NC_0:NC_999999999 [PACC]"

  esearch -db protein -query "amyloid* [PROT]" |
  elink -target pubmed -label prot_cit |
  esearch -db gene -query "apo* [GENE]" |
  elink -target pubmed -label gene_cit |
  esearch -query "(#prot_cit) AND (#gene_cit)" |
  efetch -format docsum |
  xtract -pattern DocumentSummary -element Id Title
```

1. What things are in your edirect directory/folder on the bioinformatics server?
2. How many files start with the letter **e** or **x**?
3. What directory items start with the letter **e** or **x**?

Today's Python3 exercises

Most of the files that you will need for these exercises are in the `/localdisk/data/BPSM/Lecture13` directory.

As you should already know by now, I recommend you have an outline plan for each of your scripts/programmes before you start coding and please, remember to put comments in your programme/script(s)!

It is probably much easier to use `nano &` or `vi &` to write your Python scripts, and then execute (run) the saved files in a separate bash shell (assuming of course that they have the correct file permissions and you have the correct Python3 shebang line...).

YOIKS! What happened when you did the above commands?! The ampersand `&` puts the "job" (running nano, or vi) in the "background" and runs it while you do something else. Not terribly useful in this instance, I agree... so how do you get it back? You can just type `fg` to "foreground" your job, and there you go: the nano or vi session is back!

Why might you want to use this approach? As an example, you have a whole pile of output files you want to compress, but you dont want to wait for them to finish, you just want to carry on with the rest of your script? This is how you could do it, but note that if your next process requires these files, this would NOT be a good idea, as they might not have finished yet...

```
# Compress all my fasta files as a background process...
# How many files do I have? I think it's lots...
ls *.fasta | wc -l
```

```
5309
# Compress them all and carry on without waiting for that to finish
pigz *.fasta &
mynextcommand
```

Exercise 1 : Processing DNA in a file

The file *input.txt* contains a number of DNA sequences, one per line.
Each sequence starts with the same 14 base pairs : these are from a sequencing adapter that should have been removed.

Write a Python script/programme that will :
(a) trim the adapter and write the 'cleaned' (adapter-free) sequences to a single new file AND
(b) print the length of each adapter-free sequence to the screen.

Exercise 2 : Multiple exons from genomic DNA

The file *genomic_dna2.txt* contains a section of genomic DNA.
The file *exons.txt* contains a list of start/stop positions of exons.
Each exon is on a separate line and the start and stop positions are separated by a comma.

Write a Python script/programme that will extract the exon segments, concatenate them, and write them to a new file.

Exercise 3 : Sliding windows

You need to write a Python script/programme that will generate overlapping short segments from a long string (i.e. a sliding window approach); e.g. if your input sequence was

```
abcdefghijk
```

and the window size chosen was 6, and the offset was 1, then the segments would be :

```
abcdef
bcdefg
cdefgh
defghi
efghij
fghijk
```

Using the protein-coding region from the AJ223353 NCBI sequence ('*remote_exon01.fasta*', or whatever you called it) that you generated while doing the exercises in the last lecture, write a Python programme that generates segments that are 30 bases long, with a window offset of 3.

a. Modify your Python script/programme to print each sliding window segment to the screen.
b. Modify your Python script/programme to print the percentage GC content of each sliding window segment and the sequence.
c. Modify your Python script/programme to write out the individual segments in fasta format (i.e. with an informative fasta header) into individual fasta files.
d. Modify your Python script/programme to write out the individual segments in fasta format (i.e. with an informative fasta header) into a single fasta file.
e. Modify your Python script/programme to include the partial sliding window segments that we get at the end of the sequence.

Note that all of the above 'modifications' could/should be part of a single Python script/programme! For example, you might find it easier to add processing step (a), ensure it is working properly, then try adding (b), and so on.

Exercise 4 : Size-binning DNA sequences

In the directory
`/localdisk/data/BPSM/Lecture13/`
is a collection of files whose names end in `.dna`.
Each file holds a collection of DNA sequences, one per line.

What is required:

- Write a Python script/programme which creates nine new 'size range' directories, one for sequences between 100 and 199 bases long, one for sequences between 200 and 299 bases long, etc., etc..
- Choose any one of the `.dna` files as an input file, and write out each DNA sequence in that input file to a separate file in the appropriate 'size range' directory.

Give feedback
Ask for help

---

edirect

When you installed the edirect software package, was put in the `edirect` directory in your home space.

```
cd ~/edirect
```

```
# What things are in the edirect directory/folder?
ls
```

```
accn-at-a-time          compare-uids            esummary              gbf2xml               join-into-groups-of   rchive.Linux          test-eutils
align-columns           csv2xml                 eutils                gene2range            json2xml              rcommon.sh            test-pmc-index
amino-acid-composition  custom-index           exclude-uid-lists     gff2xml               jsonl2xml             README                test-pubmed-index
archive-ncbinlp         data                    exclude-uids          gff-sort              just-top-hits         readme.pdf            test-repeat
archive-nihocc          difference-uid-lists    expand-current        gm2ranges             LICENSE               ref2pmid              theme-aliases
archive-nlmnlp          disambiguate-nucleotides extern               gm2segs               Mozilla-CA.tar.gz     refseq-nm-cds         toml2xml
archive-nmcds           download-flatfile       fetch-local           help                  nhance.sh             reorder-columns       transmute
archive-pids            download-ncbi-data      fetch-nmcds           hgvs2spdi             nquire                run-ncbi-converter    transmute.Linux
archive-pmc             download-ncbi-software  fetch-pmc             idx-affil             pair-at-a-time        scn2xml               uniq-table
archive-pubmed          download-pmc            fetch-pubmed          idx-auid              phrase-search         skip-if-file-exists   word-at-a-time
archive-taxonomy        download-pubmed         fetch-taxonomy        idx-authors           pma2apa               snp2hgvs              xbuild
args2slice              download-sequence       filter-columns        idx-doi               pma2pme               snp2tbl               xcommon.sh
asn2ref                 ds2pme                  filter-genbank        idx-errors            pmc2bioc              sort-by-length        xfetch
asn2xml                 ecollect                filter-record         idx-grant             pmc2info              sort-table            xfilter
between-two-genes       ecommon.sh              filter-stop-words     idx-journals          pm-clean              sort-uniq-count       xinfo
blst2gm                 EDIRECT.pdf             find-in-gene          idx-metadata          pm-collect            sort-uniq-count-rank  xlink
blst2tkns               edirect.py              fsa2xml               idx-pairs             pm-prepare            spdi2tbl              xml2fsa
bsmp2info               efetch                  fuse-ranges           idx-stemmed           pm-refresh            split-at-intron       xml2json
cacert.pem              efilter                 fuse-segments         idx-words             pm-setup              stream-local          xml2tbl
CA.pm                   einfo                   gbf2facds             index-extras          print-columns         stream-pubmed         xsearch
cit2pmid                elink                   gbf2fsa               index-pubmed          print-missing-subranges systematic-mutations xtract
cmd                     epost                   gbf2info              ini2xml               __pycache__           tbl2prod              xtract.Linux
combine-uid-lists       esample                 gbf2ref               intersect-uid-lists   quote-grouped-elements tbl2xml              xy-plot
combine-uids            esearch                 gbf2tbl               intersect-uids        rchive                test-edirect          yaml2xml
```

```
# How many files start with the letter e or x?
ls | egrep '^e|^x' | wc -l
29
```

```
# What file names start with the letter e or x?
ls | egrep '^e|^x'
ecollect
ecommon.sh
edirect.py
efetch
efilter
einfo
elink
epost
esample
esearch
```

```
esummary
eutils
exclude-uid-lists
exclude-uids
expand-current
extern
xbuild
xcommon.sh
xfetch
xfilter
xinfo
xlink
xml2fsa
xml2json
xml2tbl
xsearch
xtract
xtract.Linux
xy-plot
```

LOTS of help for how to use edirect software : **strongly** recommend you read it! There is also this web page that has some useful tips too...

Possible answers... to today's exercises

Linux stuff first, we need to copy the exercise-related files to our space on the server : who did it using Python?

Well done to you if you did! 👍

```
cd
mkdir Exercises/Lecture13
cd ~/Exercises/Lecture13
```

```
pwd
```

```
/home/aivens2/Exercises/Lecture13
```

```
# Check what is in the directory of provided exercise files
ls /localdisk/data/BPSM/Lecture13
```

```
exons.txt
genomic_dna2.txt
input.txt
xaa.dna
xab.dna
xac.dna
xad.dna
xae.dna
xaf.dna
xag.dna
xah.dna
xai.dna
xaj.dna
```

```
# Copy the provided exercise files into YOUR current directory
cp /localdisk/data/BPSM/Lecture13/* .
```

```
# Copy the AJ223353 coding region fasta file from YOUR Lecture12 directory
# (which should be 'one up and then one down' in the
# directory tree, in the folder Lecture12 ?)
# into the current directory
cp ../Lecture12/remote_exon01.fasta .
```

```
# Check we have what we need : three 'txt' files
ls -1 *.txt
```

```
exons.txt
genomic_dna2.txt
input.txt
```

```
# One 'fasta' file
ls -1 *.fasta
```

```
remote_exon01.fasta
```

```
# Several .dna files
ls -1 *.dna
```

```
xaa.dna
xab.dna
xac.dna
xad.dna
```

```
xae.dna
xaf.dna
xag.dna
xah.dna
xai.dna
xaj.dna
```

```
# Get some basic info on them (newline, word, byte)
```

```
wc *
```

```
    4     4    28 exons.txt
    1     1   485 genomic_dna2.txt
    5     5   282 input.txt
   10    10  4055 xaa.dna
   10    10  6603 xab.dna
   10    10  5571 xac.dna
   10    10  5337 xad.dna
   10    10  6626 xae.dna
   10    10  4991 xaf.dna
   10    10  5094 xag.dna
   10    10  5642 xah.dna
   10    10  5183 xai.dna
   10    10  6061 xaj.dna
...
```

```
head -n2 *.txt
```

```
==> exons.txt <==
5,58
72,133

==> genomic_dna2.txt <==
TCGATCGTACCGTCGACGATGCTACGATCGTCGATCGTAGTCGATCATCGATCGATCGACTGATCGATCGATCGATCGATCGATATCGAT

==> input.txt <==
ATTCGATTATAAGCTCGATCGATCGATCGATCGATCGATCGATCGATCGATC
ATTCGATTATAAGCACTGATCGATCGATCGATCGATCGATGCTATCGTCGT
```

---

1 : Processing DNA in a file

The file *input.txt* contains a number of DNA sequences, one per line.
Each sequence starts with the same 14 base pairs : these are from a sequencing adapter that should have been removed.

Write a Python script/programme that will :
(a) trim the adapter and write the 'cleaned' sequences to a single new file AND
(b) print the length of each sequence to the screen.

```
# Do a couple of things in Linux first
```

```
# Check we have what we need
```

```
wc input.txt
```

```
  5    5 282 input.txt
```

```
head input.txt
```

```
ATTCGATTATAAGCTCGATCGATCGATCGATCGATCGATCGATCGATCGATC
ATTCGATTATAAGCACTGATCGATCGATCGATCGATCGATGCTATCGTCGT
ATTCGATTATAAGCATCGATCACGATCTATCGTACGTATGCATATCGATATCGATCGTAGTC
ATTCGATTATAAGCACTATCGATGATCTAGCTACGATCGTAGCTGTA
ATTCGATTATAAGCACTAGCTAGTCTCGATGCATGATCAGCTTAGCTGATGATGCTATGCA
```

```
# Looks like the adapter might be 'ATTCGATTATAAGC' ?
```

```
# Python time...!
```

```
python3
```

```
Python 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

```
# programme to trim fixed length adapter sequences from start of each sequence
# and keep the "clean" output in a file, outputting to screen as we go
# Code written by s123456 on 28 October 2025
#.....
# Open and read the file, trimming off adapter "on the fly"
input_txt_contents = open('input.txt').read().upper().replace('ATTCGATTATAAGC','').split()
```

```
# Alternative way to do it
input_txt_contents2 = open('input.txt').read().upper().split()
len(input_txt_contents)
5
len(input_txt_contents2)
5
```

```
# Contents to a single file and screen
# Open the output pipe for writing, I've called it cleanseqs
cleanseqs = open('Cleaned_sequences.txt','w')
```

```
# Now loop through and do the outputs required
# We weren't specifically asked for fasta format...!
```

Version 1 (adapter gone already)

```
for cleanones in input_txt_contents :
    cleanseqs.write(cleanones + '\n')
    cleanones
```

```
43
'TCGATCGATCGATCGATCGATCGATCGATCGATCGATC'
38
'ACTGATCGATCGATCGATCGATGCTATCGTCGT'
49
'ATCGATCACGATCTATCGTACGTATGCATATCGATATCGATCGTAGTC'
34
'ACTATCGATGATCTAGCTACGATCGTAGCTGTA'
48
'ACTAGCTAGTCTCGATGCATGATCAGCTTAGCTGATGATGCTATGCA'
```

Version 2: adapter bit not written to file

Open an output pipe for writing, I've called it `cleanseqs2`

cleanseqs2 = open('Cleaned_sequences2.txt','w')

```
for cleanones in input_txt_contents2 :
    cleanseqs2.write(cleanones[14 :] + '\n')
    cleanones[14 :]
```

```
43
'TCGATCGATCGATCGATCGATCGATCGATCGATCGATC'
38
'ACTGATCGATCGATCGATCGATGCTATCGTCGT'
49
'ATCGATCACGATCTATCGTACGTATGCATATCGATATCGATCGTAGTC'
34
'ACTATCGATGATCTAGCTACGATCGTAGCTGTA'
48
'ACTAGCTAGTCTCGATGCATGATCAGCTTAGCTGATGATGCTATGCA'
```

"Better version" using `with`

```
with open('Cleaned_sequences3.txt','w') as cleanseqs_w :
 for cleanones in input_txt_contents :
    cleanseqs_w.write(cleanones + '\n')
    cleanones
```

```
43
'TCGATCGATCGATCGATCGATCGATCGATCGATCGATC'
38
'ACTGATCGATCGATCGATCGATGCTATCGTCGT'
49
'ATCGATCACGATCTATCGTACGTATGCATATCGATATCGATCGTAGTC'
34
'ACTATCGATGATCTAGCTACGATCGTAGCTGTA'
48
'ACTAGCTAGTCTCGATGCATGATCAGCTTAGCTGATGATGCTATGCA'
```

Are the connections closed, so we can view the file contents?

cleanseqs.closed

False

```
cleanseqs2.closed
```

```
False
```

```
cleanseqs_w.closed
```

```
True
```

```
# Close the outputs so we can view the files!
cleanseqs.close()
cleanseqs2.close()
```

```
# Check an output file
print(open('Cleaned_sequences.txt').read().rstrip())
```

```
TCGATCGATCGATCGATCGATCGATCGATCGATCGATC
ACTGATCGATCGATCGATCGATGCTATCGTCGT
ATCGATCACGATCTATCGTACGTATGCATATCGATATCGATCGTAGTC
ACTATCGATGATCTAGCTACGATCGTAGCTGTA
ACTAGCTAGTCTCGATGCATGATCAGCTTAGCTGATGATGCTATGCA
```

```
# Check our output file a different way
import subprocess
subprocess.call("cat Cleaned_sequences.txt",shell=True)
```

```
TCGATCGATCGATCGATCGATCGATCGATCGATCGATC
ACTGATCGATCGATCGATCGATGCTATCGTCGT
ATCGATCACGATCTATCGTACGTATGCATATCGATATCGATCGTAGTC
ACTATCGATGATCTAGCTACGATCGTAGCTGTA
ACTAGCTAGTCTCGATGCATGATCAGCTTAGCTGATGATGCTATGCA
0
```

---

2 : Multiple exons from genomic DNA

The file *genomic_dna2.txt* contains a section of genomic DNA.
The file *exons.txt* contains a list of start/stop positions of exons.
Each exon is on a separate line and the start and stop positions are separated by a comma.

Write a Python script/programme that will extract the exon segments, concatenate them, and write them to a new file.

```
# We could do a couple of things in Linux first
# Check we have what we need : genomic file
wc genomic_dna2.txt
```

```
  1   1 485 genomic_dna2.txt
```

```
head genomic_dna2.txt
```

```
TCGATCGTACCGTCGACGATGCTACGATCGTCGATCGTAGTCGATCATCGATCGATCGACTGA
```

```
# Check we have what we need : exons file
```

```
wc exons.txt
```

```
 4   4 28 exons.txt
```

```
head exons.txt
```

```
5,58
72,133
190,276
340,398
```

BACK IN OUR PYTHON SESSION

```
# Open and read the genomic file as a string
genomic_dna2 = open('genomic_dna2.txt').read().upper()
len(genomic_dna2)
```

```
485
```

```
# Version 1 : open and read the exons file as a list directly
exons_v1 = open('exons.txt').read().replace(',','\n').split()
exons_v1
```

```
['5', '58', '72', '133', '190', '276', '340', '398']
```

```
# Could process these taking the first/second, third/fourth etc
# using a list(range(1,10,2)) command or similar
# This approach could rapidly get too complicated, think of a better way!
```

```
# Version 2 : open and read the exons file as an rstrip() file,
# this comes in as a string
exons_v2 = open('exons.txt').read().rstrip()
exons_v2
```

```
'5,58\n72,133\n190,276\n340,398'
```

```
# Quick check
counter=0
for exons in exons_v2 :
    counter += 1
```

```
    print(counter,exons)
```

```
# Hmmm, this doesn't look like what we want...
```

```
1 5
2 ,
3 5
4 8
5

6 7
7 2
8 ,
9 1
10 3
11 3
12

13 1
14 9
15 0
16 ,
17 2
18 7
19 6
20

21 3
22 4
23 0
24 ,
25 3
26 9
27 8
```

```
type(exons_v2)
```

```
<class 'str'>
```

```
# Version 3 : open and read the exons file as an rsplit() input,
# which comes in as a list, one line per string element
```

```
exons_v3 = open('exons.txt').read().rsplit()
```

```
type(exons_v3)
```

```
<class 'list'>
```

```
exons_v3
```

```
['5,58', '72,133', '190,276', '340,398']
```

```
# Quick check
```

```
counter=0
for exons in exons_v3 :
    counter += 1
    print(counter,exons)
```

```
1 5,58
2 72,133
3 190,276
4 340,398
```

```
# This looks correct, let's use this version
counter=0
# Open pipe to file for writing output
with open('Exercise2_coding_seqence.fasta','w') as fullcoding :
# Fasta header line insertion : write to the pipe
# Then extract the coding segments, need Python3 index positions
  fullcoding.write('>Lecture13_exercise2_codingseq\n')
  for exons in exons_v3 :
    counter += 1
    startexon = int(exons.split(',')[0])-1
    endexon = int(exons.split(',')[1])
    exonwanted = genomic_dna2[startexon :endexon]
    fullcoding.write(exonwanted)
    regionsummary = 'Exon'+str(counter)+' '+str(exons)+' runs from index position '+str(st
    print(regionsummary,'\n\t',exonwanted)
```

```
31
54
Exon1 5,58 runs from index position 4 upto but not including 58.
         TCGTACCGTCGACGATGCTACGATCGTCGATCGTAGTCGATCATCGATCGATCG
62
Exon2 72,133 runs from index position 71 upto but not including 133.
         TCGATCGATCGATATCGATCGATATCATCGATGCATCGATCATCGATCGATCGATCGATCGA
87
Exon3 190,276 runs from index position 189 upto but not including 276.
         TCGATCGATCGATCGTAGCTAGCTAGCTAGATCGATCATCATCGTAGCTAGCTCGACTAGCTACGTACGATCGA
59
Exon4 340,398 runs from index position 339 upto but not including 398.
         ACGATCGATCGATCGATCGATCGATCGATCGATCGATCGTAGCTAGCTACGATCG
```

```
# Closed the file?
fullcoding.closed
True
```

```
# Coding region file check
print(open('Exercise2_coding_seqence.fasta').read())
```

```
>Lecture13_exercise2_codingseq
TCGTACCGTCGACGATGCTACGATCGTCGATCGTAGTCGATCATCGATCGATCGTCGATCGATCGATATCGATCGATATCATC
```

NOTE : we could also have used something like this to do it :

```
for exonlocs in open('exons.txt') :
   start=int(exonlocs.split(",")[0])-1
   stop=int(exonlocs.split(",")[1].rstrip())
   exonwanted = genomic_dna2[start :stop]


    and so on
```

---

3 : Sliding windows

You need to write a Python script/programme that will generate overlapping short segments from a long string (i.e. a sliding window approach).

Using the protein-coding region from the AJ223353 NCBI sequence ('*remote_exon01.fasta*', or whatever you called it) that you generated while doing the exercises in the last lecture, write a Python programme that generates segments that are 30 bases long, with a window offset of 3.

a. Modify your Python script/programme to print each sliding window segment to the screen.
b. Modify your Python script/programme to print the percentage GC content of each sliding window segment and the sequence.
c. Modify your Python script/programme to write out the individual segments in fasta format (i.e. with an informative fasta header) into individual fasta files.
d. Modify your Python script/programme to write out the individual segments in fasta format (i.e. with an informative fasta header) into a single fasta file.
e. Modify your Python script/programme to include the partial sliding window segments that we get at the end of the sequence.

 # Copying the file

```
import os, shutil
```

```
shutil.copy("../Lecture12/remote_exon01.fasta", ".")
```

```
'./remote_exon01.fasta'
```

```
sorted(os.listdir())
```

```
['Cleaned_sequences.txt', 'Cleaned_sequences2.txt', 'Cleaned_sequences3
  'Exercise2_coding_seqence.fasta', 'exons.txt', 'genomic_dna2.txt', 'i
  'remote_exon01.fasta', 'xaa.dna', 'xab.dna', 'xac.dna', 'xad.dna', 'x
  'xaf.dna', 'xag.dna', 'xah.dna', 'xai.dna', 'xaj.dna']
```

 # What is in the file?

```
open('remote_exon01.fasta').read().split()
```

```
['>AJ223353_exon01_length381', 'ATGCCTGAACCTACCAAGTCTGCTCCTGCCCCAAAGAAGGGCTCCAAGAAC
```

```
# Read in the sequence as a string, ignoring the first list element as it is the fasta header line
AJ223353_seq = open('remote_exon01.fasta').read().split()[1]
len(AJ223353_seq)
381
```

```
# Check the end of the sequence : should be a stop codon
AJ223353_seq[-3:]
'TAA'
# Double-check we know how indexing works
len(AJ223353_seq[0:30])
30
```

```
# Set the windowing parameters required
windowsize = 30
offset = 3
```

```
# Generate a numeric range for positions for the window starts
segment_starts = list(range(0,len(AJ223353_seq)-windowsize+1,offset))
```

```
# Alternative (better) range to also get the smaller segments at the end
segment_starts = list(range(0,len(AJ223353_seq),offset))
segment_starts
[0, 3, 6, 9, 12, 15, 18, 21, 24, 27, 30, 33,
...
309, 312, 315, 318, 321, 324, 327, 330, 333, 336, 339, 342, 345,
348, 351, 354, 357, 360, 363, 366, 369, 372, 375, 378]
```

```
# Create and then close a file object to take all the segment sequences
basic_fasta_header = '_window' + str(windowsize) + '_offset' + str(offset)
```

```
alloutfilename = 'AJ223353_coding_all' + basic_fasta_header + '.fasta'
with open(alloutfilename, 'w') as allsegments :
 allsegments.write('')
# Create a blank list to hold the segments
 segments_made = []
# Create a possible yes/no switching variable, i.e. a conditional
 fileswanted = 1
# The for loop
```

```
 for seg_bits in segment_starts :
     tempseq = AJ223353_seq[seg_bits :seg_bits+windowsize].upper()
     segments_made = segments_made + [tempseq]
# percentage GC content, convert float to integer value
     tempGC = int(100 * (tempseq.count('G') + tempseq.count('C'))/len(tempseq) )
# make a fasta header line
     descriptionline = 'AJ223353_coding_'+str(len(segments_made))+'_'+tempseq+basic_fasta_
     fastaheader = '>'+descriptionline
     print(len(segments_made),'\t',tempseq,'\t',tempGC)
# Question : do we want files written? Answer will be either True or False ('maybe' is NOT
     if fileswanted == 1 :
# open the segment fastafiles
        with open(descriptionline+'.fasta', 'w') as segmentfile :
# output to files
          segmentfile.write(fastaheader+'\n'+tempseq)
          allsegments.write(fastaheader+'\n'+tempseq+'\n\n')
     else :
        allsegments.close()
```

segmentfile.closed

True

allsegments.closed

True

 # This is the output when files NOT requested : segment ID, sequence, GC

```
0
1        ATGCCTGAACCTACCAAGTCTGCTCCTGCC          56
2        CCTGAACCTACCAAGTCTGCTCCTGCCCCA          60
3        GAACCTACCAAGTCTGCTCCTGCCCCAAAG          56
4        CCTACCAAGTCTGCTCCTGCCCCAAAGAAG          56
5        ACCAAGTCTGCTCCTGCCCCAAAGAAGGGC          60
6        AAGTCTGCTCCTGCCCCAAAGAAGGGCTCC          60
7        TCTGCTCCTGCCCCAAAGAAGGGCTCCAAG          60

.......... etc
116      ACCAAGGCCGTCACCAAGTACACCAGTTCC          56
117      AAGGCCGTCACCAAGTACACCAGTTCCAAG          53
118      GCCGTCACCAAGTACACCAGTTCCAAGTAA          50
119      GTCACCAAGTACACCAGTTCCAAGTAA     44
120      ACCAAGTACACCAGTTCCAAGTAA        41
121      AAGTACACCAGTTCCAAGTAA   38
122      TACACCAGTTCCAAGTAA      38
123      ACCAGTTCCAAGTAA         40
124      AGTTCCAAGTAA    33
125      TCCAAGTAA       33
126      AAGTAA          16
127      TAA     0
```

 # Outside the loop, print the first 10 segments, one per line

print('\n'.join(segments_made[0:10]))

```
ATGCCTGAACCTACCAAGTCTGCTCCTGCC
CCTGAACCTACCAAGTCTGCTCCTGCCCCA
GAACCTACCAAGTCTGCTCCTGCCCCAAAG
CCTACCAAGTCTGCTCCTGCCCCAAAGAAG
```

```
ACCAAGTCTGCTCCTGCCCCAAAGAAGGGC
AAGTCTGCTCCTGCCCCAAAGAAGGGCTCC
TCTGCTCCTGCCCCAAAGAAGGGCTCCAAG
GCTCCTGCCCCAAAGAAGGGCTCCAAGAAG
CCTGCCCCAAAGAAGGGCTCCAAGAAGGCG
GCCCCAAAGAAGGGCTCCAAGAAGGCGGTG
```

```
# Print the first 3 segments, separated by the word 'then'
```

```
print(' then '.join(segments_made[0:3]))
```

```
ATGCCTGAACCTACCAAGTCTGCTCCTGCC then CCTGAACCTACCAAGTCTGCTCCTGCCCCA then GAACCTACCAAGTCTGCT
```

```
# Does the combined fasta file output look correct?
```

```
# Using a loop makes this easy
```

```
for these in 1,2,3 :
    open(alloutfilename).read().split('>')[these]
```

```
'AJ223353_coding_1_ATGCCTGAACCTACCAAGTCTGCTCCTGCC_window30_offset3\nATGCCTGAACCTACCAAGTCTG
'AJ223353_coding_2_CCTGAACCTACCAAGTCTGCTCCTGCCCCA_window30_offset3\nCCTGAACCTACCAAGTCTGCTC
'AJ223353_coding_3_GAACCTACCAAGTCTGCTCCTGCCCCAAAG_window30_offset3\nGAACCTACCAAGTCTGCTCCTG
```

```
# Better still, take advantage of the \n\n between each sequence in the combined fasta file...
```

```
print('\n'.join(open(alloutfilename).read().split('\n\n')[0:3]))
```

```
>AJ223353_coding_1_ATGCCTGAACCTACCAAGTCTGCTCCTGCC_window30_offset3
ATGCCTGAACCTACCAAGTCTGCTCCTGCC
>AJ223353_coding_2_CCTGAACCTACCAAGTCTGCTCCTGCCCCA_window30_offset3
CCTGAACCTACCAAGTCTGCTCCTGCCCCA
>AJ223353_coding_3_GAACCTACCAAGTCTGCTCCTGCCCCAAAG_window30_offset3
GAACCTACCAAGTCTGCTCCTGCCCCAAAG
```

```
# Or just use the file object, this will process the whole file
```

```
my_file = open(alloutfilename)
for line in my_file :
    print(line.rstrip('\n\n'))
```

```
>AJ223353_coding_1_ATGCCTGAACCTACCAAGTCTGCTCCTGCC_window30_offset3
ATGCCTGAACCTACCAAGTCTGCTCCTGCC
>AJ223353_coding_2_CCTGAACCTACCAAGTCTGCTCCTGCCCCA_window30_offset3
CCTGAACCTACCAAGTCTGCTCCTGCCCCA
>AJ223353_coding_3_GAACCTACCAAGTCTGCTCCTGCCCCAAAG_window30_offset3
GAACCTACCAAGTCTGCTCCTGCCCCAAAG
```

```
# What files do I have in my directory?
```

```
dir()
```

```
['AJ223353_seq', '__annotations__', '__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__', '__warni
```

```
# NO, that shows the things I have in my Python session/"bubble", not what I have saved!
```

```
# We need to be able to 'talk' to the operating system (os), luckily we have that os module to do that
```

```
# Have I already imported the os module?
```

```
'os' in dir()
```

```
True
```

```
os.getcwd()
```

```
'/localdisk/home/aivens2/Exercises/Lecture13'
```

```
# We can actually remove os functionality from our "bubble"
```

```
del os
```

```
os.getcwd()
```

```
Traceback (most recent call last) :
  File '<stdin>', line 1, in <module>
NameError : name 'os' is not defined
```

```
'os' in dir()
```

```
False
```

```
# Import os
```

```
import os
```

```
# Now we can get current working directory again
```

```
os.getcwd()
```

```
'/localdisk/home/aivens2/Exercises/Lecture13'
```

```
# How many files/directories are there, visible from where we are?
```

```
len(os.listdir())
```

```
146
```

```
# Show me the files
```

```
os.listdir()
```

```
['exons.txt', 'genomic_dna2.txt', 'input.txt', 'xaa.dna',
.....
  'AJ223353_coding_8_GCTCCTGCCCCAAAGAAGGGCTCCAAGAAG_window30_offset3.fasta',
  'AJ223353_coding_100_CGCCTGCTGCTTCCGGGGGAGCTGGCCAAG_window30_offset3.fasta',
  'AJ223353_coding_125_TCCAAGTAA_window30_offset3.fasta',
  'AJ223353_coding_94_GAGATCCAGACGGCCGTGCGCCTGCTGCTT_window30_offset3.fasta',
  'AJ223353_coding_104_CCGGGGGAGCTGGCCAAGCACGCCGTGTCG_window30_offset3.fasta',
```

```
.....
  'AJ223353_coding_87_CGCTCGACCATCACCTCCAGGGAGATCCAG_window30_offset3.fasta',
  'AJ223353_coding_109_AAGCACGCCGTGTCGGAGGGCACCAAGGCC_window30_offset3.fasta']
```

```
os.listdir()[0]
```

```
'exons.txt'
```

```
os.listdir()[1]
```

```
'genomic_dna2.txt'
```

```
# Show me the "dna" files
```

```
os.listdir('*.dna')
```

```
Traceback (most recent call last) :
  File '<stdin>', line 1, in <module>
FileNotFoundError : [Errno 2] No such file or directory : '*.dna'
```

```
# Need to import another module if we want to use the * wildcard...
```

```
import glob
```

```
# What functionality does this module have?
```

```
# DOT TAB TAB
```

```
glob.
```

```
gglob.contextlib          glob.glob(              glob.has_magic(          glob.magic_check
glob.escape(              glob.glob0(             glob.iglob(              glob.magic_check_b
glob.fnmatch              glob.glob1(             glob.itertools           glob.os
```

```
glob.glob('*.dna')
```

```
['xaa.dna', 'xad.dna', 'xab.dna', 'xaj.dna', 'xac.dna', 'xag.dna', 'xaf.dna',
'xah.dna', 'xae.dna', 'xai.dna']
```

```
len(glob.glob('*.dna'))
```

```
10
```

```
glob.glob('*.txt')
```

```
glob.glob('*.txt') ['exons.txt', 'genomic_dna2.txt', 'input.txt',
'Cleaned_sequences.txt', 'Cleaned_sequences2.txt',
'Cleaned_sequences3.txt']
```

---

4 : Size binning

```
# In Linux shell
```

```
pwd
```

```
/home/aivens2/Exercises/Lecture13
```

```
# What files are there that we might need?
```

```
ls *.dna
```

```
xaa.dna  xab.dna  xac.dna  xad.dna  xae.dna  xaf.dna  xag.dna  xah.dna  xai.dna  xaj.dna
```

```
# How many sequences are there in each/all the files?
```

```
wc -l *.dna
```

```
  10 xaa.dna
  10 xab.dna
  10 xac.dna
  10 xad.dna
  10 xae.dna
  10 xaf.dna
  10 xag.dna
  10 xah.dna
  10 xai.dna
  10 xaj.dna
 100 total
```

```
# In Linux, make a sub-directory for the sequence files
```

```
mkdir dna_files
```

```
# Move the dna files to the dna_files directory that we have just made
```

```
mv *.dna ./dna_files
```

```
# Have a quick look at one file to see what the format is
```

```
head -n5 ./dna_files/xaa.dna
```

```
ctgtaacttagttggctctttcgtagcccattgtcgggctagctatttcactcccgcgggggtctccgcgtggatggacgtgc
gagggtatcctccttttggaccagtgtcccgctcgttcccaatatgggcactgttatcgcgtctataaaagacgcggtggatc
ctgttcagcgggtgctgggcttgatctaacagcacaactcttagatggcgtcacgcctctgcctaatggctaggaaagtcttc
cctgcattcgccaaatacccactcgttgatgtgcaaagaagctagcaggttcataaggtttacgggacagttatccgttgccc
cacaatcagcgcgaacatactcgaatgtaaacctgagacacccacggtcctagccggacccgaaagctaggaaaacgtgatct
```

```
# Start Python
```

```
python3
```

```
# Import the various modules we think we might need
```

```
import os, sys, subprocess, shutil
```

```
# Get a list of the files we might want to process
```

```
# We need the path : directory and the file name.
```

```
# This loop would open each file
```

```
for file_name in os.listdir('dna_files/') :
    if file_name.endswith('.dna') :
        f'Reading sequences from {file_name}'
        dna_file = open('dna_files/' + file_name)
# This loop then looks at each line and gets the length
# Note the indentation...
        for line in dna_file :
            dna = line.rstrip('\n')
            length = len(dna)
            print(f'\tFound a DNA sequence with length {str(length)}' )
```

```
'Reading sequences from xaa.dna'
        Found a DNA sequence with length 333
        Found a DNA sequence with length 283
        Found a DNA sequence with length 380
        Found a DNA sequence with length 115
        Found a DNA sequence with length 753
        Found a DNA sequence with length 764
        Found a DNA sequence with length 234
        Found a DNA sequence with length 117
        Found a DNA sequence with length 906
        Found a DNA sequence with length 160
'Reading sequences from xab.dna'
        Found a DNA sequence with length 833
        Found a DNA sequence with length 390
        Found a DNA sequence with length 355
.... etc
```

```
# Now add some code that will try each size bin in turn to see if the sequence fits.
```

```
# We'll use a range to check the bounds on each size bin
```

```
# Go through each file in the directory
```

```
for file_name in  sorted(os.listdir('dna_files')) :
# Check if the file name ends with .dna
  if file_name.endswith('.dna') :
    print('Reading sequences from ' + file_name)
# Open the file and process each line
    dna_file = open('dna_files/' + file_name)
# Calculate the sequence length
    for line in dna_file :
      dna = line.rstrip('\n')
      length = len(dna)
      print('\tsequence length is ' + str(length))
# Go through each size bin and check if the sequence belongs in it
      for bin_lower in list(range(100,1000,100)) :
        bin_upper = bin_lower + 99
        if length >= bin_lower and length <= bin_upper :
          print('\t\tbin is ' + str(bin_lower) + ' to ' + str(bin_upper))
```

```
Reading sequences from xaa.dna
        sequence length is 333
                bin is 300 to 399
        sequence length is 283
```

```
                bin is 200 to 299
        sequence length is 380
                bin is 300 to 399
        sequence length is 115
                bin is 100 to 199
        sequence length is 753
                bin is 700 to 799
...
```

We will need to create some new directories so that we can write

each DNA sequence to the correct one based on the directory name

```
for bin_lower in list(range(100,1000,100)) :
    bin_upper = bin_lower + 99
    bin_directory_name = str(bin_lower) + '_' + str(bin_upper)
    os.mkdir(bin_directory_name)
```

What directories do we have now? Don't want to use `os.listdir()` !

len(os.listdir())

146

Could do this, using `isdir`

```
for name in os.listdir(".") :
   if os.path.isdir(name) :
     print (name)
```

```
100_199
200_299
300_399
400_499
500_599
600_699
700_799
800_899
900_999
dna_files
```

Or use `os.walk()` to generate the names in a directory tree

mydirs = os.walk('.')

mydirs

<generator object walk at 0x7faf4fb7f9e0>

mydirs[0]

```
Traceback (most recent call last) :
  File "<stdin>", line 1, in <module>
TypeError : 'generator' object is not subscriptable
```

Access the contents with a `for` loop

sorted([x[0] for x in mydirs])

```
['.', './100_199', './200_299', './300_399', './400_499', './500_599',
  './600_699', './700_799', './800_899', './900_999', './dna_files']
```

```
# Can we use the generator object again? No, would need to recreate it.
```

```
sorted([x[0] for x in mydirs])
```

```
[]
```

```
# Let's run our programme now
# Re-create a new directory for each size bin, essentially deleting anything there previously
for bin_lower in list(range(100,1000,100)) :
    bin_upper = bin_lower + 99
    bin_directory_name = str(bin_lower) + '_' + str(bin_upper)
    shutil.rmtree(bin_directory_name)
    os.mkdir(bin_directory_name)
```

```
# create a variable to hold an arbitrary sequence identifier number
seq_number = 1
```

```
# Go through each file in the directory
for file_name in os.listdir('dna_files') :
# Check if the file name ends with .dna
  if file_name.endswith('.dna') :
    print('Reading sequences from ' + file_name)
# Open the file and process each line
    dna_file = open('dna_files/' + file_name)
    for line in dna_file :
# Calculate the sequence length
      dna = line.rstrip('\n')
      length = len(dna)
      print('\tsequence length is ' + str(length))
# Go through each size bin and check if the sequence belongs in it
      for bin_lower in list(range(100,1000,100)) :
        bin_upper = bin_lower + 99
        if length >= bin_lower and length <= bin_upper :
          print('\t\tbin is ' + str(bin_lower) + ' to ' + str(bin_upper))
          bin_directory_name = str(bin_lower) + '_' + str(bin_upper)
          output_path = bin_directory_name + '/' + str(seq_number) + '.dna'
          output = open(output_path, 'w')
          output.write(dna)
          output.close()
# Increment the sequence number
          seq_number = seq_number+1
```

```
Reading sequences from xaa.dna
        sequence length is 333
                bin is 300 to 399
333
        sequence length is 283
                bin is 200 to 299
283
        sequence length is 380
                bin is 300 to 399
```

```
380
....
```

```
# Finally, see what we been done
os.listdir('dna_files')
['xaa.dna', 'xab.dna', 'xac.dna', 'xad.dna', 'xae.dna', 'xaf.dna', 'xag.dna',
'xah.dna', 'xai.dna', 'xaj.dna']
```

```
# Look in one of the size directories we made
sorted(os.listdir('100_199'))
['10.dna', '22.dna', '27.dna', '4.dna', '67.dna', '73.dna', '8.dna', '89.dna',
'91.dna', '97.dna']
```

```
# Open one of the files in a size directory we made
print(open('100_199/4.dna').read())
cctgcattcgccaaatacccactcgttgatgtgcaaagaagctagcaggttcataaggtttacgggacagttatccg
```

```
# Was it the right size for that directory? Yes!
len(open('100_199/4.dna').read())
115
```

etc...

PHEW!

---

git and GitHub time!

```
git st
On branch master
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    ../Lecture04/CodeFiles.tar.gz
        modified:   ../Lecture06/run_this_script_v1

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        ../BN.out
        ../BN.sh
        ../Lecture04/Als_ICA/
        ../Lecture04/all_the_things_I_did
```

```
        ../Lecture04/outs/
        ../Lecture05/Country.Mozambique.details
        ../Lecture07/Cosmoscarta.nuc.fa
        ../Lecture07/Cosmoscarta.nuc.gis
        ../Lecture07/myfileofaccessions.txt
        ../Lecture12/AJ223353.fasta
        ../Lecture12/AJ223353.genbank
        ../Lecture12/AJ223353_noheader.fasta
        ../Lecture12/AJ223353_noheader.fasta2
        ../Lecture12/AJ223353_noheader.fasta3
        ../Lecture12/All_exons.fasta
        ../Lecture12/All_noncodings.fasta
        ../Lecture12/local_exon01.fasta
        ../Lecture12/local_exon02.fasta
        ../Lecture12/local_intron01.fasta
        ../Lecture12/my_dna.txt
        ../Lecture12/plain_genomic_seq.txt
        ../Lecture12/quick_blast_check.out
        ../Lecture12/remote_exon01.fasta
        ../Lecture12/remote_noncoding01.fasta
        ../Lecture12/remote_noncoding02.fasta
        ../Lecture12/shutilversion.txt
        ./

no changes added to commit (use "git add" and/or "git commit -a")
```

```
# You might want to add other files?
```

```
ls *.py
```

```
mypie.py
```

```
git add *.py
```

```
git commit -m "Scripts from the python lists and loops lecture 2"
```

```
[master b880335] Scripts from the python lists and loops lecture 2
 1 file changed, 35 insertions(+)
 create mode 100644 Lecture13/mypie.py
```

```
# Which link to GitHub did we want to use for pushing?
```

```
git remote
```

```
AllLectures
Lecture07
origin
```

```
# Push our local repository to the AllLectures repo at GitHub
```

```
# This time we will save our credentials for our PERSONAL account
# This is NOT secure...
```

```
git config credential.helper store
```

```

```

```
git push AllLectures master
```

```
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 256 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 1.38 KiB | 708.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/B123456-2121/BPSM_LectureStuff.git
   e444ef8..b880335  master -> master
```

Give feedback
Ask for help

git st

```
On branch master
Changes not staged for commit:
  (use "git add/rm ..." to update what will be committed)
  (use "git restore ..." to discard changes in working directory)
        deleted:    ../Lecture04/CodeFiles.tar.gz
        modified:   ../Lecture06/run_this_script_v1

Untracked files:
  (use "git add ..." to include in what will be committed)
        ../BN.out
        ../BN.sh
        ../Lecture04/Als_ICA/
        ../Lecture04/all_the_things_I_did
        ../Lecture04/outs/
        ../Lecture05/Country.Mozambique.details
        ../Lecture07/Cosmoscarta.nuc.fa
        ../Lecture07/Cosmoscarta.nuc.gis
        ../Lecture07/myfileofaccessions.txt
        ../Lecture12/AJ223353.fasta
        ../Lecture12/AJ223353.genbank
        ../Lecture12/AJ223353_noheader.fasta
        ../Lecture12/AJ223353_noheader.fasta2
        ../Lecture12/AJ223353_noheader.fasta3
        ../Lecture12/All_exons.fasta
        ../Lecture12/All_noncodings.fasta
        ../Lecture12/local_exon01.fasta
        ../Lecture12/local_exon02.fasta
        ../Lecture12/local_intron01.fasta
        ../Lecture12/my_dna.txt
        ../Lecture12/plain_genomic_seq.txt
        ../Lecture12/quick_blast_check.out
        ../Lecture12/remote_exon01.fasta
        ../Lecture12/remote_noncoding01.fasta
        ../Lecture12/remote_noncoding02.fasta
        ../Lecture12/shutilversion.txt
        100_199/
        200_299/
        300_399/
        400_499/
        500_599/
        600_699/
        700_799/
        800_899/
        900_999/
        AJ223353_coding_100_CGCCTGCTGCTTCCGGGGGAGCTGGCCAAG_window30_offset3.fasta
        AJ223353_coding_101_CTGCTGCTTCCGGGGGAGCTGGCCAAGCAC_window30_offset3.fasta
        AJ223353_coding_102_CTGCTTCCGGGGGAGCTGGCCAAGCACGCC_window30_offset3.fasta
        AJ223353_coding_103_CTTCCGGGGGAGCTGGCCAAGCACGCCGTG_window30_offset3.fasta
        AJ223353_coding_104_CCGGGGGAGCTGGCCAAGCACGCCGTGTCG_window30_offset3.fasta
        AJ223353_coding_105_GGGGAGCTGGCCAAGCACGCCGTGTCGGAG_window30_offset3.fasta
        AJ223353_coding_106_GAGCTGGCCAAGCACGCCGTGTCGGAGGGC_window30_offset3.fasta
        AJ223353_coding_107_CTGGCCAAGCACGCCGTGTCGGAGGGCACC_window30_offset3.fasta
        AJ223353_coding_108_GCCAAGCACGCCGTGTCGGAGGGCACCAAG_window30_offset3.fasta
        AJ223353_coding_109_AAGCACGCCGTGTCGGAGGGCACCAAGGCC_window30_offset3.fasta
        AJ223353_coding_10_GCCCCAAAGAAGGGCTCCAAGAAGGCGGTG_window30_offset3.fasta
        AJ223353_coding_110_CACGCCGTGTCGGAGGGCACCAAGGCCGTC_window30_offset3.fasta
        AJ223353_coding_111_GCCGTGTCGGAGGGCACCAAGGCCGTCACC_window30_offset3.fasta
        AJ223353_coding_112_GTGTCGGAGGGCACCAAGGCCGTCACCAAG_window30_offset3.fasta
        AJ223353_coding_113_TCGGAGGGCACCAAGGCCGTCACCAAGTAC_window30_offset3.fasta
        AJ223353_coding_114_GAGGGCACCAAGGCCGTCACCAAGTACACC_window30_offset3.fasta
        AJ223353_coding_115_GGCACCAAGGCCGTCACCAAGTACACCAGT_window30_offset3.fasta
        AJ223353_coding_116_ACCAAGGCCGTCACCAAGTACACCAGTTCC_window30_offset3.fasta
        AJ223353_coding_117_AAGGCCGTCACCAAGTACACCAGTTCCAAG_window30_offset3.fasta
        AJ223353_coding_118_GCCGTCACCAAGTACACCAGTTCCAAGTAA_window30_offset3.fasta
        AJ223353_coding_119_GTCACCAAGTACACCAGTTCCAAGTAA_window30_offset3.fasta
        AJ223353_coding_11_CCAAAGAAGGGCTCCAAGAAGGCGGTGACT_window30_offset3.fasta
```

AJ223353_coding_120_ACCAAGTACACCAGTTCCAAGTAA_window30_offset3.fasta
AJ223353_coding_121_AAGTACACCAGTTCCAAGTAA_window30_offset3.fasta
AJ223353_coding_122_TACACCAGTTCCAAGTAA_window30_offset3.fasta
AJ223353_coding_123_ACCAGTTCCAAGTAA_window30_offset3.fasta
AJ223353_coding_124_AGTTCCAAGTAA_window30_offset3.fasta
AJ223353_coding_125_TCCAAGTAA_window30_offset3.fasta
AJ223353_coding_126_AAGTAA_window30_offset3.fasta
AJ223353_coding_127_TAA_window30_offset3.fasta
AJ223353_coding_12_AAGAAGGGCTCCAAGAAGGCGGTGACTAAG_window30_offset3.fasta
AJ223353_coding_13_AAGGGCTCCAAGAAGGCGGTGACTAAGGCT_window30_offset3.fasta
AJ223353_coding_14_GGCTCCAAGAAGGCGGTGACTAAGGCTCAG_window30_offset3.fasta
AJ223353_coding_15_TCCAAGAAGGCGGTGACTAAGGCTCAGAAG_window30_offset3.fasta
AJ223353_coding_16_AAGAAGGCGGTGACTAAGGCTCAGAAGAAG_window30_offset3.fasta
AJ223353_coding_17_AAGGCGGTGACTAAGGCTCAGAAGAAGGAC_window30_offset3.fasta
AJ223353_coding_18_GCGGTGACTAAGGCTCAGAAGAAGGACGGG_window30_offset3.fasta
AJ223353_coding_19_GTGACTAAGGCTCAGAAGAAGGACGGGAAG_window30_offset3.fasta
AJ223353_coding_1_ATGCCTGAACCTACCAAGTCTGCTCCTGCC_window30_offset3.fasta
AJ223353_coding_20_ACTAAGGCTCAGAAGAAGGACGGGAAGAAG_window30_offset3.fasta
AJ223353_coding_21_AAGGCTCAGAAGAAGGACGGGAAGAAGCGC_window30_offset3.fasta
AJ223353_coding_22_GCTCAGAAGAAGGACGGGAAGAAGCGCAAG_window30_offset3.fasta
AJ223353_coding_23_CAGAAGAAGGACGGGAAGAAGCGCAAGCGC_window30_offset3.fasta
AJ223353_coding_24_AAGAAGGACGGGAAGAAGCGCAAGCGCAGC_window30_offset3.fasta
AJ223353_coding_25_AAGGACGGGAAGAAGCGCAAGCGCAGCCGC_window30_offset3.fasta
AJ223353_coding_26_GACGGGAAGAAGCGCAAGCGCAGCCGCAAG_window30_offset3.fasta
AJ223353_coding_27_GGGAAGAAGCGCAAGCGCAGCCGCAAGGAG_window30_offset3.fasta
AJ223353_coding_28_AAGAAGCGCAAGCGCAGCCGCAAGGAGAGC_window30_offset3.fasta
AJ223353_coding_29_AAGCGCAAGCGCAGCCGCAAGGAGAGCTAT_window30_offset3.fasta
AJ223353_coding_2_CCTGAACCTACCAAGTCTGCTCCTGCCCCA_window30_offset3.fasta
AJ223353_coding_30_CGCAAGCGCAGCCGCAAGGAGAGCTATTCA_window30_offset3.fasta
AJ223353_coding_31_AAGCGCAGCCGCAAGGAGAGCTATTCAGTG_window30_offset3.fasta
AJ223353_coding_32_CGCAGCCGCAAGGAGAGCTATTCAGTGTAT_window30_offset3.fasta
AJ223353_coding_33_AGCCGCAAGGAGAGCTATTCAGTGTATGTG_window30_offset3.fasta
AJ223353_coding_34_CGCAAGGAGAGCTATTCAGTGTATGTGTAC_window30_offset3.fasta
AJ223353_coding_35_AAGGAGAGCTATTCAGTGTATGTGTACAAG_window30_offset3.fasta
AJ223353_coding_36_GAGAGCTATTCAGTGTATGTGTACAAGGTG_window30_offset3.fasta
AJ223353_coding_37_AGCTATTCAGTGTATGTGTACAAGGTGCTG_window30_offset3.fasta
AJ223353_coding_38_TATTCAGTGTATGTGTACAAGGTGCTGAAG_window30_offset3.fasta
AJ223353_coding_39_TCAGTGTATGTGTACAAGGTGCTGAAGCAG_window30_offset3.fasta
AJ223353_coding_3_GAACCTACCAAGTCTGCTCCTGCCCCAAAG_window30_offset3.fasta
AJ223353_coding_40_GTGTATGTGTACAAGGTGCTGAAGCAGGTC_window30_offset3.fasta
AJ223353_coding_41_TATGTGTACAAGGTGCTGAAGCAGGTCCAT_window30_offset3.fasta
AJ223353_coding_42_GTGTACAAGGTGCTGAAGCAGGTCCATCCC_window30_offset3.fasta
AJ223353_coding_43_TACAAGGTGCTGAAGCAGGTCCATCCCGAC_window30_offset3.fasta
AJ223353_coding_44_AAGGTGCTGAAGCAGGTCCATCCCGACACC_window30_offset3.fasta
AJ223353_coding_45_GTGCTGAAGCAGGTCCATCCCGACACCGGC_window30_offset3.fasta
AJ223353_coding_46_CTGAAGCAGGTCCATCCCGACACCGGCATC_window30_offset3.fasta
AJ223353_coding_47_AAGCAGGTCCATCCCGACACCGGCATCTCT_window30_offset3.fasta
AJ223353_coding_48_CAGGTCCATCCCGACACCGGCATCTCTTCC_window30_offset3.fasta
AJ223353_coding_49_GTCCATCCCGACACCGGCATCTCTTCCAAG_window30_offset3.fasta
AJ223353_coding_4_CCTACCAAGTCTGCTCCTGCCCCAAAGAAG_window30_offset3.fasta
AJ223353_coding_50_CATCCCGACACCGGCATCTCTTCCAAGGCA_window30_offset3.fasta
AJ223353_coding_51_CCCGACACCGGCATCTCTTCCAAGGCAATG_window30_offset3.fasta
AJ223353_coding_52_GACACCGGCATCTCTTCCAAGGCAATGGGG_window30_offset3.fasta
AJ223353_coding_53_ACCGGCATCTCTTCCAAGGCAATGGGGATC_window30_offset3.fasta
AJ223353_coding_54_GGCATCTCTTCCAAGGCAATGGGGATCATG_window30_offset3.fasta
AJ223353_coding_55_ATCTCTTCCAAGGCAATGGGGATCATGAAT_window30_offset3.fasta
AJ223353_coding_56_TCTTCCAAGGCAATGGGGATCATGAATTCC_window30_offset3.fasta
AJ223353_coding_57_TCCAAGGCAATGGGGATCATGAATTCCTTC_window30_offset3.fasta
AJ223353_coding_58_AAGGCAATGGGGATCATGAATTCCTTCGTC_window30_offset3.fasta
AJ223353_coding_59_GCAATGGGGATCATGAATTCCTTCGTCAAC_window30_offset3.fasta
AJ223353_coding_5_ACCAAGTCTGCTCCTGCCCCAAAGAAGGGC_window30_offset3.fasta
AJ223353_coding_60_ATGGGGATCATGAATTCCTTCGTCAACGAC_window30_offset3.fasta
AJ223353_coding_61_GGGATCATGAATTCCTTCGTCAACGACATC_window30_offset3.fasta
AJ223353_coding_62_ATCATGAATTCCTTCGTCAACGACATCTTC_window30_offset3.fasta
AJ223353_coding_63_ATGAATTCCTTCGTCAACGACATCTTCGAG_window30_offset3.fasta
AJ223353_coding_64_AATTCCTTCGTCAACGACATCTTCGAGCGC_window30_offset3.fasta
AJ223353_coding_65_TCCTTCGTCAACGACATCTTCGAGCGCATC_window30_offset3.fasta
AJ223353_coding_66_TTCGTCAACGACATCTTCGAGCGCATCGCA_window30_offset3.fasta
AJ223353_coding_67_GTCAACGACATCTTCGAGCGCATCGCAGGC_window30_offset3.fasta
AJ223353_coding_68_AACGACATCTTCGAGCGCATCGCAGGCGAG_window30_offset3.fasta

```
AJ223353_coding_69_GACATCTTCGAGCGCATCGCAGGCGAGGCT_window30_offset3.fasta
AJ223353_coding_6_AAGTCTGCTCCTGCCCCAAAGAAGGGCTCC_window30_offset3.fasta
AJ223353_coding_70_ATCTTCGAGCGCATCGCAGGCGAGGCTTCC_window30_offset3.fasta
AJ223353_coding_71_TTCGAGCGCATCGCAGGCGAGGCTTCCCGC_window30_offset3.fasta
AJ223353_coding_72_GAGCGCATCGCAGGCGAGGCTTCCCGCCTG_window30_offset3.fasta
AJ223353_coding_73_CGCATCGCAGGCGAGGCTTCCCGCCTGGCG_window30_offset3.fasta
AJ223353_coding_74_ATCGCAGGCGAGGCTTCCCGCCTGGCGCAT_window30_offset3.fasta
AJ223353_coding_75_GCAGGCGAGGCTTCCCGCCTGGCGCATTAC_window30_offset3.fasta
AJ223353_coding_76_GGCGAGGCTTCCCGCCTGGCGCATTACAAC_window30_offset3.fasta
AJ223353_coding_77_GAGGCTTCCCGCCTGGCGCATTACAACAAG_window30_offset3.fasta
AJ223353_coding_78_GCTTCCCGCCTGGCGCATTACAACAAGCGC_window30_offset3.fasta
AJ223353_coding_79_TCCCGCCTGGCGCATTACAACAAGCGCTCG_window30_offset3.fasta
AJ223353_coding_7_TCTGCTCCTGCCCCAAAGAAGGGCTCCAAG_window30_offset3.fasta
AJ223353_coding_80_CGCCTGGCGCATTACAACAAGCGCTCGACC_window30_offset3.fasta
AJ223353_coding_81_CTGGCGCATTACAACAAGCGCTCGACCATC_window30_offset3.fasta
AJ223353_coding_82_GCGCATTACAACAAGCGCTCGACCATCACC_window30_offset3.fasta
AJ223353_coding_83_CATTACAACAAGCGCTCGACCATCACCTCC_window30_offset3.fasta
AJ223353_coding_84_TACAACAAGCGCTCGACCATCACCTCCAGG_window30_offset3.fasta
AJ223353_coding_85_AACAAGCGCTCGACCATCACCTCCAGGGAG_window30_offset3.fasta
AJ223353_coding_86_AAGCGCTCGACCATCACCTCCAGGGAGATC_window30_offset3.fasta
AJ223353_coding_87_CGCTCGACCATCACCTCCAGGGAGATCCAG_window30_offset3.fasta
AJ223353_coding_88_TCGACCATCACCTCCAGGGAGATCCAGACG_window30_offset3.fasta
AJ223353_coding_89_ACCATCACCTCCAGGGAGATCCAGACGGCC_window30_offset3.fasta
AJ223353_coding_8_GCTCCTGCCCCAAAGAAGGGCTCCAAGAAG_window30_offset3.fasta
AJ223353_coding_90_ATCACCTCCAGGGAGATCCAGACGGCCGTG_window30_offset3.fasta
AJ223353_coding_91_ACCTCCAGGGAGATCCAGACGGCCGTGCGC_window30_offset3.fasta
AJ223353_coding_92_TCCAGGGAGATCCAGACGGCCGTGCGCCTG_window30_offset3.fasta
AJ223353_coding_93_AGGGAGATCCAGACGGCCGTGCGCCTGCTG_window30_offset3.fasta
AJ223353_coding_94_GAGATCCAGACGGCCGTGCGCCTGCTGCTT_window30_offset3.fasta
AJ223353_coding_95_ATCCAGACGGCCGTGCGCCTGCTGCTTCCG_window30_offset3.fasta
AJ223353_coding_96_CAGACGGCCGTGCGCCTGCTGCTTCCGGGG_window30_offset3.fasta
AJ223353_coding_97_ACGGCCGTGCGCCTGCTGCTTCCGGGGGAG_window30_offset3.fasta
AJ223353_coding_98_GCCGTGCGCCTGCTGCTTCCGGGGGAGCTG_window30_offset3.fasta
AJ223353_coding_99_GTGCGCCTGCTGCTTCCGGGGGAGCTGGCC_window30_offset3.fasta
AJ223353_coding_9_CCTGCCCCAAAGAAGGGCTCCAAGAAGGCG_window30_offset3.fasta
AJ223353_coding_all_window30_offset3.fasta
Cleaned_sequences.txt
Cleaned_sequences2.txt
Cleaned_sequences3.txt
Exercise2_coding_seqence.fasta
dna_files/
exons.txt
genomic_dna2.txt
input.txt
remote_exon01.fasta
```

Probably want to remove some of all those fasta files, (or at least tar gzip them!)?

tar cvfz Lecture13.tar.gz *offset3* --remove-files

```
AJ223353_coding_100_CGCCTGCTGCTTCCGGGGGAGCTGGCCAAG_window30_offset3.fasta
AJ223353_coding_101_CTGCTGCTTCCGGGGGAGCTGGCCAAGCAC_window30_offset3.fasta
AJ223353_coding_102_CTGCTTCCGGGGGAGCTGGCCAAGCACGCC_window30_offset3.fasta
AJ223353_coding_103_CTTCCGGGGGAGCTGGCCAAGCACGCCGTG_window30_offset3.fasta
...
```

ls

```
100_199  400_499  700_799  Cleaned_sequences2.txt  dna_files                       genomic
200_299  500_599  800_899  Cleaned_sequences3.txt  Exercise2_coding_seqence.fasta  input.t
300_399  600_699  900_999  Cleaned_sequences.txt   exons.txt                       Lecture
```

```
# Ooops, just realised that I didn't want that commit message,
# it was the THIRD python lecture...
# we can amend our message!
git commit --amend -m "Scripts from the python lists and loops lecture 3"
```

```
# If you don't use the -m notation, the defined editor
# will start here (edit as required, save and exit)
[master 8f58f4c] Scripts from the python lists and loops lecture 3
 Date: Tue Oct 28 11:29:01 2025 +0000
 1 file changed, 97 insertions(+)
 create mode 100644 Lecture13/mypie.py
```

```
# Try to push again with new message
git push AllLectures master
To https://github.com/B123456-2121/BPSM_LectureStuff.git
 ! [rejected]        master -> master (non-fast-forward)
error: failed to push some refs to 'https://github.com/B123456-2121/BPSM_LectureStuff.git'
hint: Updates were rejected because the tip of your current branch is behind
hint: its remote counterpart. Integrate the remote changes (e.g.
hint: 'git pull ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

```
git lo
8f58f4c (HEAD -> master) Scripts from the python lists and loops lecture 3
e444ef8 Scripts from the second BPSM python lecture
6d6f5e7 My python scripts from the first BPSM python lecture
eae65c6 (Lecture07/master) edirect seqs related stuff added
eea4f5e (Lec_6_branch) Kept bash script output files from Lecture 06
fbe5cc0 Added run_this_script_v1 from BPSM Lecture06
ce2c15a nematode BLASTDB stuff added
19cdd2d Added the example_people_data.tsv file from Lecture03
137c3db Decided to keep blastoutput2.out
08d379b Decided to keep myinputfile.tsv
c9b5b18 First attempt at parsing BLAST data
bbf825c My first awk script, Lecture05
68f27fb There were conflicts with someotherfile.sh, kept all changes
83629cb (Version1.2) Final version of the someotherfile.sh shell script prior to merge
0b5425b (tag: v2.0) Updated contents of tar, coolscript
5d8aeea I am not sure cool script is actually any good
bbd36e7 Added my_cool_script
ea60357 Third version of the someotherfile.sh shell script
b6ae4a7 Updated version of tar
ce230e1 (tag: v1.2) Second version of the someotherfile.sh shell script
a5add7c Added the someotherfile.sh shell script
101b700 Additional motif files, and updated tar file
d21cb3d Adding intial tar file
59d1756 CodeFiles all done and tar zipped
6dd85a8 First file added
```

```
# This is how we do it.... and no username/token asked for!
git push --force-with-lease AllLectures master
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 256 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 1.39 KiB | 1.39 MiB/s, done.
```

```
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/B123456-2121/BPSM LectureStuff.git
 + b880335...8f58f4c master -> master (forced update)
```

```
# ALWAYS keep checking
git lo
```

```
8f58f4c (HEAD -> master, AllLectures/master) Scripts from the python lists and loops lectu
e444ef8 Scripts from the second BPSM python lecture
6d6f5e7 My python scripts from the first BPSM python lecture
eae65c6 (Lecture07/master) edirect seqs related stuff added
eea4f5e (Lec_6_branch) Kept bash script output files from Lecture 06
fbe5cc0 Added run_this_script_v1 from BPSM Lecture06
ce2c15a nematode BLASTDB stuff added
19cdd2d Added the example_people_data.tsv file from Lecture03
137c3db Decided to keep blastoutput2.out
08d379b Decided to keep myinputfile.tsv
c9b5b18 First attempt at parsing BLAST data
bbf825c My first awk script, Lecture05
68f27fb There were conflicts with someotherfile.sh, kept all changes
83629cb (Version1.2) Final version of the someotherfile.sh shell script prior to merge
0b5425b (tag: v2.0) Updated contents of tar, coolscript
5d8aeea I am not sure cool script is actually any good
bbd36e7 Added my_cool_script
ea60357 Third version of the someotherfile.sh shell script
b6ae4a7 Updated version of tar
ce230e1 (tag: v1.2) Second version of the someotherfile.sh shell script
a5add7c Added the someotherfile.sh shell script
101b700 Additional motif files, and updated tar file
d21cb3d Adding intial tar file
59d1756 CodeFiles all done and tar zipped
6dd85a8 First file added
```

We decided un-encrypted credentials (user name and git token) wasn't a very good idea (insecure, can only have one set, etc.), so we removed them from the cache, meaning we WILL have to type them in next time...

```
# Clearing credentials...
git credential-cache exit
```

START OF BRIEF DETOUR....

If you DO want to have credentials stored, it is much safer to use an "SSH key". Instructions for doing that are given here on this web page.

END OF BRIEF DETOUR....

Have your say...

The "Postbox of Truth": is your chance to give anonymous "in course" feedback/comments. Your comments are completely anonymous and confidential, so say what you think, irrespective of whether the comments are positive or otherwise! You can use this multiple times too: I use it throughout the course to enable you to give context-relevant feedback/make suggestions.

Just click the image!

Sources used